Amazon EC2 — Complete DevOps Interview Notes

> **Level:** Beginner to advanced (4–6 years of experience target)
>
> **Roles covered:** DevOps Engineer, Senior DevOps Engineer, SRE, Cloud Engineer, Platform Engineer, Linux/Systems Engineer
>
> **Style:** Simple English, pointwise notes, production scenarios, interview-ready answers

How to use these notes

1. Read sections 1–14 to build the mental model.
2. Practise sections 15–16 (CLI, Terraform, incidents) in a sandbox AWS account.
3. Drill sections 20–21 (Q&A and answer framework) until you can answer without reading.
4. Use section 22 for last-day revision.

Conventions

- `<placeholder>` values and names such as `prod-web-01` are examples. No real keys, passwords, tokens or account IDs appear here.
- **[MANDATORY]** marks a control you should never skip in production. **[OPTIONAL]** marks a design choice that depends on requirements and cost.
- Defaults, quotas, prices, instance generations and newer features change. Where that applies, the note says "verify in the current AWS documentation".
- Amazon EC2 Auto Scaling has its own complete notes (`ec2-auto-scaling.md`). Section 13 here is only a short summary and points there for depth.

Table of Contents

1. What Is Amazon EC2
2. How EC2 Works and Core Components
3. AMI: Amazon Machine Image
4. Instance Types and Right-Sizing
5. AWS EC2 Pricing Options
6. Instance Lifecycle: Stop, Reboot, Terminate, Hibernate
7. Storage: EBS and Instance Store
8. Networking for EC2
9. Security Groups vs Network ACLs
10. EC2 Access: Key Pairs, Session Manager and Instance Connect
11. User Data, cloud-init and the Instance Metadata Service
12. IAM Roles and Instance Profiles
13. Launch Templates and Auto Scaling (Summary)
14. Monitoring, Status Checks and Recovery
15. Hands-On: CLI and Terraform
16. Production Troubleshooting Scenarios
17. Security, Reliability and Best Practices
18. Comparison Tables
19. Enterprise Reference Architecture
20. Interview Questions and Answers
21. Interview Answer Framework
22. Quick Revision

---

1. What Is Amazon EC2

1.1 Definition

1. **Amazon Elastic Compute Cloud (EC2)** lets you create and manage **virtual servers (instances)** in the AWS cloud.

2. You choose the operating system (through an **AMI**), CPU and memory (through an **instance type**), storage, networking and permissions.
3. AWS owns the physical hardware, the data centre and the hypervisor layer (the **Nitro System** on current generations). **You own the guest operating system, the software on it, patching, access and configuration.** This is the **shared responsibility model**.
4. You pay for what you use, according to the purchasing option (section 5).

1.2 Why it is needed

1. **Elasticity:** add or remove capacity in minutes.
2. **Scalability:** add more instances (scale out) or use bigger instances (scale up).
3. **Choice:** Linux or Windows, many CPU, memory, storage and GPU shapes.
4. **Automation:** launch with the AWS CLI, SDKs, CloudFormation or Terraform.
5. **High availability:** spread instances over several Availability Zones (AZs).
6. **Security:** IAM, security groups, KMS encryption, private networking, Session Manager.
7. **Pay-as-you-go:** per-second billing for most Linux instances (verify the current minimum and rules for your OS).

1.3 Example

You must host a Java application.

Step	Action
1	Launch an EC2 instance from an Amazon Linux AMI
2	Attach a role so the app can read one S3 bucket
3	Allow only the load balancer to reach port 8080
4	Install Java and the application (user data or a baked AMI)
5	Monitor it, and later put it in an Auto Scaling group

1.4 When EC2 is not the best fit

1. Event-driven short tasks: consider AWS Lambda.
2. Containers without server management: consider ECS or EKS with Fargate.
3. Fully managed databases: consider RDS or DynamoDB.
4. Static websites: consider S3 with CloudFront.
5. EC2 is the right choice when you need **OS control**, special software, GPUs, licences tied to servers, long-running stateful processes or custom networking.

1.5 Interview pointers

Q1 (what and why).

---

2. How EC2 Works and Core Components

2.1 The launch flow

```
1. Choose AMI            -> OS + software image
2. Choose instance type  -> vCPU, memory, network, accelerators
3. Network + security    -> VPC, subnet (one AZ), public IP?, security group, IAM role
4. Storage + bootstrap   -> EBS volumes, user data, key pair or SSM access
5. Launch                -> pending -> running
6. Connect and operate   -> SSH or Session Manager, deploy, monitor
```

An instance lives in **one Region, one AZ and one subnet** of a **VPC**. Storage, network rules and identity are configured **separately** and can be changed independently (mostly).

2.2 Architecture

```
Region
 +------------------------------- VPC 10.0.0.0/16 --------------------------------+
 |  AZ-a                                      AZ-b                                 |
 |  +------------------ subnet 10.0.1.0/24 ---+  +------------- subnet 10.0.2.0/24 ---+ |
 |  |  EC2 instance                          |  |  EC2 instance                      | |
 |  |   - ENI (private IP, security groups)  |  |   - ENI                            | |
 |  |   - Root EBS volume (from AMI)         |  |   - Root EBS volume                | |
 |  |   - IAM role via instance profile      |  |   - IAM role via instance profile  | |
 |  +----------------------------------------+  +------------------------------------+ |
 +-------------------------------------------------------------------------------------+
 Control plane: EC2 API, CloudTrail, CloudWatch, Systems Manager, KMS
```

2.3 Core components

Component	Purpose
AMI	Template with OS and software used to launch instances
Instance type	CPU, memory, network and storage capabilities
EBS	Persistent block storage
Instance store	Temporary storage on the host (supported types only)
Key pair	Public/private key for SSH (Linux) or password decryption (Windows)
Security group	Stateful virtual firewall on network interfaces
VPC / subnet	Your network and the AZ-bound IP range inside it
IAM role + instance profile	Temporary AWS permissions for software on the instance
User data	Bootstrap script or cloud-init config run at launch
Elastic IP	Static public IPv4 address you can re-associate
ENI	Elastic Network Interface: virtual network card with IPs and SGs
Launch template	Versioned, reusable launch settings
Auto Scaling group	Keeps the desired number of instances and scales them
Instance metadata (IMDS)	Local service at `169.254.169.254` giving instance information and role credentials

2.4 Key points

1. **An IAM role is not an SSH key.** The role authorises **AWS API calls** made by software on the instance. The key pair authenticates a **person** to the operating system.
2. A **subnet belongs to exactly one AZ**. An instance can only be placed in a subnet of that AZ.
3. Most settings are fixed at launch, but you can later change security groups, IAM profile, instance type (after stopping), termination protection, EBS volume size and more.

2.5 Interview pointers

Q2 (launch flow and components), Q10 (IAM role vs key pair).

---

3. AMI: Amazon Machine Image

3.1 What it is

1. An **AMI** is a template used to launch instances. It contains a **root volume snapshot** (or a template for the root volume), **launch permissions** and a **block device mapping**.
2. An AMI is **Regional**. To use it in another Region, **copy** it there. Copies of encrypted AMIs need a KMS key in the destination.
3. An AMI is **not** an instance. It is the image used to create instances.

3.2 Types of AMIs

Type	Notes
AWS-provided	Amazon Linux 2023, Ubuntu, Windows and others published by AWS or vendors via AWS
AWS Marketplace	Third-party images, often with software licence charges
Community	Shared publicly by anyone. **Treat as untrusted** until reviewed
Custom (your own)	Built by your organisation from a hardened base. Best for production

3.3 Why use custom AMIs

1. **Consistency:** 30 web servers all start identical.
2. **Speed:** software is already installed, so instances become healthy faster (important for Auto Scaling).
3. **Security:** a hardened, scanned image with agents (monitoring, SSM) pre-installed.
4. **Fewer boot-time failures:** no dependency on package mirrors at launch.

3.4 Golden AMI pipeline (production pattern)

```
Base AMI (latest AWS-provided)
   -> EC2 Image Builder or Packer build (install packages, agents, hardening)
   -> Tests and vulnerability scan (for example Amazon Inspector, CIS checks)
   -> Tag + version the AMI, share to target accounts/Regions
   -> Update launch template version -> roll out (instance refresh / blue-green)
   -> Deprecate old AMIs after a retention period
```

1. **[MANDATORY]** No passwords, private keys, access keys or tokens inside an AMI.
2. Patch by **building a new AMI and replacing instances** (immutable approach), not by patching in place, when you run fleets.
3. Use **SSM Parameter Store public parameters** to find the latest Amazon Linux AMI ID instead of hard-coding it (example in section 15.4).
4. Use **AMI deprecation** and **Recycle Bin** features to manage old images and protect against accidental deletion. Verify the current options.

3.5 Creating an AMI from an instance

1. `create-image` takes snapshots of the attached volumes. By default it **reboots** the instance for file-system consistency. `--no-reboot` avoids the reboot but the image may be only crash-consistent.
2. For databases, quiesce or use application-aware backups. An AMI is not an application-consistent backup by itself.
3. AMIs are backed by snapshots, which cost storage. **Deregister the AMI and delete its snapshots** when no longer needed (deregistering alone leaves the snapshots).

3.6 AMI vs instance

AMI	EC2 instance
Image/template	Running or stopped virtual server
Used to create instances	Runs your workload
Shared or copied with permissions	Has its own lifecycle, IPs and state

3.7 Common mistakes

1. Launching unreviewed community AMIs.
2. Baking secrets or SSH keys into an image.
3. Forgetting that AMIs are Regional.
4. Never refreshing the AMI, so new instances come up unpatched.
5. Deleting an AMI but leaving the snapshots (cost) or the reverse (breaking launch templates).

3.8 Interview pointers

Q3 (AMI), Q24 (AMI pipeline and patching).

---

4. Instance Types and Right-Sizing

4.1 Naming

Example: `m7i.large`

Part	Meaning
`m`	Family (purpose)
`7`	Generation (newer usually gives better price/performance)
`i`	Attribute: `i` Intel, `a` AMD, `g` AWS Graviton (Arm), `d` local NVMe disk, `n` network-optimised, `e` extra memory or storage
`large`	Size within the family

Available types, sizes and specifications depend on the Region and AZ. Verify with `aws ec2 describe-instance-types`.

4.2 Main families

Family	Best use	Example workload
General purpose (T, M, Mac)	Balanced CPU and memory	Web servers, small to medium apps
Compute optimised (C)	High CPU per memory	Batch, encoding, high-performance web, game servers
Memory optimised (R, X, z)	Large memory	In-memory caches and databases, analytics
Storage optimised (I, D, H)	High local disk throughput or IOPS	NoSQL, data warehouses, log processing
Accelerated computing (P, G, Inf, Trn)	GPUs and ML chips	Training, inference, graphics
HPC (Hpc)	Tightly coupled compute	Simulation

4.3 Burstable instances (T family)

1. T instances give a **baseline CPU** and earn **CPU credits** when below baseline. They spend credits to burst above it.
2. If credits run out, the instance is **throttled to baseline** (**standard** mode) or **charged for surplus credits** (**unlimited** mode). Check the default for your generation and set it explicitly.
3. Watch `CPUCreditBalance` and `CPUSurplusCreditBalance` in CloudWatch.
4. They suit spiky, low-average workloads. **They are a poor fit for steady high CPU**, where an M or C type is cheaper and more predictable.

4.4 Architectures and the Nitro System

1. **Graviton (Arm64)** instances (names with `g`) usually cost less for the same work, but need **Arm-compatible AMIs, binaries and container images**.
2. **x86_64** (Intel and AMD) has the widest software compatibility.
3. The **Nitro System** offloads networking, storage (EBS) and security to dedicated hardware. It gives near bare-metal performance, **ENA** networking, NVMe EBS volumes and stronger isolation. Most current-generation types run on Nitro.
4. **Bare metal** instances (`*.metal`) give direct hardware access (for example licences or custom hypervisors).

4.5 How to select an instance type

1. Define the workload: CPU-bound, memory-bound, I/O-bound or network-bound.
2. Measure real utilisation (CloudWatch, the CloudWatch agent for memory, `top`, `vmstat`, `iostat`, load tests).
3. Check **network bandwidth** and **EBS bandwidth** limits of the type, not just vCPU and RAM.
4. Check OS, architecture and licence compatibility.
5. Start modestly, **right-size with data** (AWS Compute Optimizer gives recommendations), and re-check after releases.
6. Prefer **newer generations** and consider **Graviton** after testing.

4.6 Changing the instance type

1. Stop the instance, change the type, start it. The new type must be compatible with the AMI (virtualisation, architecture, ENA/NVMe drivers).
2. This is a short outage per instance. For fleets, change the launch template and roll out with instance refresh.
3. Public IPv4 (non-Elastic) changes after stop/start.

4.7 Interview scenario

CPU is high on a production server. First, check CloudWatch `CPUUtilization`, the `top` process list, the load average, `iowait` (is it really disk?) and application logs. Decide whether it is CPU-bound, then fix the code or query, scale out, or scale up. For T types, check credit balance first.

4.8 Interview pointers

Q4 (families and selection), Q19 (burstable credits), Q21 (Nitro, ENA, EBS-optimised).

---

5. AWS EC2 Pricing Options

These notes are in simple English, with examples, memory tricks, and interview answers.

5.1 EC2 Pricing Options

5.1.1 On-Demand Instances

Definition: Pay for EC2 compute capacity as you use it. No long-term commitment is required.

- No upfront commitment.
- Pay based on usage and the instance pricing model.
- You can start or stop instances as needed.
- Best for unpredictable workloads and short-term use.
- Usually more expensive than commitment-based options for steady workloads.

Example: You need an EC2 server for testing today and might not need it next month. Choose On-Demand.

Memory trick: On-Demand = Use it, pay for it.

Interview answer: On-Demand Instances let us pay for compute capacity without making a long-term commitment. They are suitable for development, testing, and workloads with unpredictable demand.

5.1.2 Savings Plans

Definition: Commit to a specific amount of eligible compute spending per hour for 1 or 3 years to receive discounted rates.

- Commitment is measured in dollars per hour, not a fixed number of EC2 instances.
- You pay the committed amount even if your eligible usage is lower.
- Eligible usage above the commitment is charged at applicable rates.
- Best for workloads with predictable baseline usage.
- Two main types: Compute Savings Plans and EC2 Instance Savings Plans.

Example: You commit to $10 per hour of eligible compute usage for one year. If you use enough eligible services, you can receive discounted rates up to that commitment.

Memory trick: Savings Plans = Commit to spending, save on eligible usage.

Interview answer: Savings Plans reduce compute costs in exchange for a 1- or 3-year hourly spending commitment. Compute Savings Plans offer more flexibility across eligible compute services, while EC2 Instance Savings Plans provide deeper discounts with narrower instance-family and regional flexibility.

5.1.3 Reserved Instances (RIs)

Definition: Make a 1-year or 3-year commitment to receive discounted pricing for eligible EC2 usage.

- Standard RIs generally offer larger discounts but less flexibility.
- Convertible RIs offer more flexibility to exchange reservations for different configurations.
- Payment options can include No Upfront, Partial Upfront, and All Upfront, depending on the offering.
- RIs are primarily a billing discount, not a guarantee that capacity will be available.
- Best for workloads that run regularly for a long time.

Example: Your production application needs an eligible EC2 configuration 24/7 for the next three years. A Reserved Instance may reduce the cost.

Memory trick: Reserved Instance = Commit for a discount.

Interview answer: Reserved Instances provide discounted pricing in exchange for a one- or three-year commitment. Unlike On-Demand Capacity Reservations, they do not automatically guarantee that EC2 capacity will be available when needed.

5.1.4 Spot Instances

Definition: Use spare AWS EC2 capacity at a discount, but AWS can interrupt the instance when that capacity is needed elsewhere.

- Can be significantly cheaper than On-Demand.
- AWS can interrupt Spot Instances, generally providing a two-minute interruption notice when possible.
- Best for fault-tolerant, flexible workloads.
- Not ideal for workloads that cannot tolerate interruption.
- Can be used with Auto Scaling groups and mixed-instance fleets.

Example: You need 100 servers to process a large batch of data. If a server is interrupted, the job can resume on another instance. Spot Instances can reduce costs.

Memory trick: Spot = Cheap, but interruptible.

Interview answer: Spot Instances let us use spare EC2 capacity at discounted prices. I would use them for batch processing, CI/CD workers, and fault-tolerant workloads, while using On-Demand capacity for critical workloads that require predictable availability.

5.1.5 Dedicated Hosts and Dedicated Instances

These options provide EC2 capacity on hardware dedicated to a single customer, but they are not identical.

A. Dedicated Host

Definition: A physical server dedicated to your use, with visibility into the host and its underlying hardware capacity.

- You can control instance placement on the host.
- Useful for software licenses tied to physical sockets or cores.
- Supports host-level visibility and certain host-affinity requirements.
- Can cost more and requires more capacity planning.

Example: Your company has a software license tied to physical CPU sockets and needs to track the host hardware.

Memory trick: Dedicated Host = Your dedicated physical server.

B. Dedicated Instance

Definition: EC2 instances run on hardware dedicated to a single customer, but you have less visibility and control over the physical host.

- Other customers' instances are not placed on that dedicated hardware.
- You do not get the same host-level placement control as with Dedicated Hosts.
- Useful when dedicated hardware is required without managing host placement.

Example: Your company needs EC2 instances on hardware dedicated to it but does not need to manage physical host placement.

Memory trick: Dedicated Instance = Dedicated hardware, less host control.

Interview answer: Dedicated Hosts provide a physical server with host-level visibility and placement control, while Dedicated Instances run on hardware dedicated to one customer without the same level of physical host management.

5.1.6 On-Demand Capacity Reservations

Definition: Reserve EC2 capacity in a specific Availability Zone for as long as you need it, without a long-term commitment.

- Helps ensure capacity is available when you launch matching instances.
- Applies to a specific Availability Zone and instance configuration.
- No one- or three-year term is required.
- You pay the applicable On-Demand rate for reserved capacity, whether or not you use it.

- It is a capacity reservation, not automatically a discount.
- Can be combined with eligible Savings Plans or Reserved Instances for billing discounts.

Example: Your company must launch 10 EC2 instances in a particular Availability Zone for an important release. You create a Capacity Reservation to secure the required capacity.

Memory trick: Capacity Reservation = Reserve capacity, not a discount.

Interview answer: On-Demand Capacity Reservations reserve EC2 capacity in a specified Availability Zone without a long-term commitment. They are useful for critical workloads that need capacity assurance, and they can be combined with eligible billing discounts.

5.1.7 Quick Comparison Table

Option	Main benefit	Commitment	Main limitation
On-Demand	Flexibility	None	Usually higher cost
Savings Plans	Lower compute cost	1 or 3 years	Hourly spending commitment
Reserved Instances	Discounted eligible usage	1 or 3 years	Less flexibility for Standard RIs
Spot Instances	Very low cost	No long-term commitment	Can be interrupted
Dedicated Host	Physical host control	Depends on purchase option	Higher cost and management
Dedicated Instance	Dedicated hardware	Depends on purchase option	Less host-level control
On-Demand Capacity Reservations	Capacity assurance	No long-term commitment	Pay even if unused

5.1.8 Which One Should You Choose? — Interview Scenarios

Scenario	Best fit
Temporary development server	On-Demand
Stable compute usage across eligible services	Compute Savings Plans
Stable EC2 usage with a known configuration	EC2 Instance Savings Plans or Reserved Instances
Interruptible batch processing	Spot Instances
Software license tied to physical hardware	Dedicated Host
Dedicated hardware without host-level control	Dedicated Instance
Critical launch requiring capacity in a specific AZ	On-Demand Capacity Reservation

5.1.9 Easy Memory Revision

Option	One-line memory
On-Demand	No commitment
Savings Plans	Commit spending to save
Reserved Instances	Commit usage for a discount
Spot Instances	Cheap but interruptible
Dedicated Host / Instance	Hardware dedicated to one customer
Capacity Reservation	Ensure capacity in a specific AZ

> **Important interview distinction:** Savings Plans and Reserved Instances are primarily pricing commitments; Capacity Reservations address capacity availability. Spot Instances prioritize cost savings but may be interrupted.

5.2 Spot Instances — Essentials

AWS EC2 — Spot Essentials and Cost Optimization (Simple Notes)

5.2.1 Spot is cheap but interruptible

- Spot Instances are cheaper than On-Demand Instances.

- AWS can stop or terminate them when it needs the capacity.
- Design applications to handle interruptions.
- Use checkpoints, retries, and graceful shutdown.

Example: If a batch job stops, it should restart from where it left off.

5.2.2 Two-minute interruption notice

- AWS generally provides a 2-minute warning before interrupting a Spot Instance.
- You can check the notice using instance metadata:

```
/latest/meta-data/spot/instance-action
```

- EventBridge can also help detect Spot interruption events.
- A rebalance recommendation may arrive earlier, warning that an instance is at increased risk of interruption.

5.2.3 Use multiple instance types and Availability Zones

- Do not depend on only one instance type or one AZ.
- Use multiple instance types and AZs to improve availability.
- Prefer the `price-capacity-optimized` allocation strategy, which balances price and the availability of Spot capacity.

5.2.4 Use Auto Scaling Groups or EC2 Fleet

- Run Spot Instances through an Auto Scaling Group or EC2 Fleet.
- These services help manage multiple instances and replace capacity when possible.
- Avoid depending on a single Spot Instance.

5.2.5 Avoid Spot for critical, non-recoverable workloads

Do not use Spot as the only instance for:

- A database primary that cannot tolerate interruption.
- A critical service running on only one instance.
- A workload that cannot recover after an unexpected stop.

Use On-Demand or another suitable reliable option for critical workloads.

Memory trick: Spot = Cheap + Interruptible + Recoverable design.

Interview answer: "Spot Instances provide low-cost EC2 capacity but can be interrupted by AWS. I use them for fault-tolerant workloads, diversify instance types and Availability Zones, and run them through Auto Scaling Groups or EC2 Fleet. I also design applications with checkpoints, retries, and graceful shutdown."

5.3 AWS EC2 Cost Optimization

Cost-saving method	Simple explanation
1. Right-size instances	Choose the correct EC2 size based on CPU and memory usage. Use AWS Compute Optimizer and CloudWatch metrics to identify oversized instances.
2. Stop idle instances	Automatically stop development and testing servers when nobody needs them. Use EventBridge Scheduler or Instance Scheduler. Stopped instances still incur EBS storage charges.
3. Delete unused EBS volumes and old snapshots	Find unattached EBS volumes and unnecessary old snapshots. Delete them only after confirming they are not needed for recovery.
4. Use GP3 instead of GP2	GP3 volumes can offer lower storage costs with independently configurable IOPS and throughput. Check workload performance before migrating.
5. Use AWS Graviton	Choose compatible Graviton-based EC2 instances for potentially better price-performance. Check application and software compatibility first.
6. Reduce public IPv4 costs	AWS charges for public IPv4 addresses, including addresses in use. Release unused Elastic IPs and avoid unnecessary public IP addresses.
7. Optimize NAT Gateway and data transfer	Use VPC endpoints for S3 and DynamoDB where appropriate to reduce NAT Gateway processing charges. Keep traffic within the same Availability Zone when practical, while considering availability and architecture.

8. Use tags and budgets	Apply cost-allocation tags to identify spending by project or environment. Use AWS Budgets and Cost Anomaly Detection to monitor unexpected costs.

5.3.1 Simple real-world example

Suppose your development environment has 20 EC2 instances, and nobody uses them at night.

1. Schedule them to stop after office hours.
2. Right-size oversized instances.
3. Delete unused EBS volumes after verification.
4. Review NAT Gateway and public IPv4 charges.
5. Set AWS Budgets alerts to track spending.

Interview answer: "I optimize AWS costs by right-sizing EC2 instances using Compute Optimizer and CloudWatch, scheduling non-production instances to stop during idle hours, cleaning up unused EBS resources, evaluating GP3 and Graviton, optimizing NAT Gateway and data-transfer costs, and using cost-allocation tags, AWS Budgets, and Cost Anomaly Detection."

Memory trick: Right-size → Stop → Clean up → Optimize → Monitor.

---

6. Instance Lifecycle: Stop, Reboot, Terminate, Hibernate

6.1 States

```
pending -> running -> stopping -> stopped -> pending -> running
                  \-> shutting-down -> terminated
running -> rebooting -> running            (reboot keeps the instance in "running")
```

6.2 Operations compared

Operation	What happens	Key points
Reboot	OS restarts, same host	Keeps instance ID, private IP, public IP, instance-store data
Stop	Powers off an **EBS-backed** instance	No compute charge. EBS charges continue. Instance-store data is **lost**. Public IPv4 (non-Elastic) is released. Usually moves to a new host on start
Start	Starts a stopped instance	Gets a new public IPv4 unless an Elastic IP is attached. Private IP stays
Hibernate	Saves RAM to the encrypted root EBS volume, then stops	Needs a supported type, enabled at launch, encrypted root volume, RAM size limits. Faster resume with application state kept
Terminate	Deletes the instance	Cannot be undone. Root volume is deleted if `DeleteOnTermination` is true (the default for the root volume). Other volumes are kept by default

6.3 Important details

1. **Instance store data is lost** on stop, hibernate, terminate and underlying hardware failure. It **survives a reboot**.
2. **Termination protection** (`DisableApiTermination`) blocks API/console termination. It does **not** stop an Auto Scaling group from terminating instances (use scale-in protection) or the OS from shutting down (see `InstanceInitiatedShutdownBehavior`).
3. **Stop protection** (`DisableApiStop`) also exists. Verify details.
4. An **Auto Scaling group replaces** instances it considers unhealthy or that you stopped. Do not stop members manually. Use **Standby** (see Auto Scaling notes).
5. Billing: **running** instances are billed. **Stopped** instances are not billed for compute but their EBS volumes, snapshots and Elastic IPs may be. **Terminated** instances stop billing.

6.4 Production tip

Before any stop or terminate: confirm the workload, Auto Scaling membership, backups (snapshots/AMI), data on instance store, Elastic IP and DNS dependencies, and the recovery plan.

6.5 Interview pointers

Q5 (stop vs reboot vs terminate), Q22 (hibernate).

---

7. Storage: EBS and Instance Store

7.1 Amazon EBS (Elastic Block Store)

1. **EBS** provides **network-attached block volumes**. They live in **one AZ** and attach to instances **in that AZ**.
2. They persist independently of the instance (according to `DeleteOnTermination`).
3. Use for root volumes, application data and databases. Volumes can be encrypted with KMS.
4. Some volumes support **Multi-Attach** (io1/io2) for specific clustered applications.

7.2 EBS volume types

Type	Category	Notes
gp3	General purpose SSD	Default choice. Baseline IOPS and throughput independent of size; you can provision more
gp2	General purpose SSD	IOPS tied to size (about 3 IOPS per GiB) with burst credits. Legacy
io2 (and io2 Block Express)	Provisioned IOPS SSD	High, consistent IOPS and durability for I/O-intensive databases
io1	Provisioned IOPS SSD	Older provisioned IOPS type
st1	Throughput-optimised HDD	Large sequential workloads (logs, big data). **Cannot be a boot volume**
sc1	Cold HDD	Infrequently accessed data. **Cannot be a boot volume**

Limits (sizes, maximum IOPS and throughput) change. Verify in the current documentation. IOPS and throughput are also capped by the **instance type's EBS bandwidth**.

7.3 Instance store

1. **Local disks physically attached to the host** (types with `d` or storage families such as I and D).
2. Very low latency and high IOPS. **Temporary**: data is lost on stop, hibernate, terminate or host failure.
3. Use for caches, buffers, scratch space and data replicated elsewhere.

Feature	EBS	Instance store
Persistence	Persists across stop/start	Temporary
Independent lifecycle	Yes (attach, detach)	No
Snapshots	Yes	Not as EBS snapshots
Encryption	KMS	Encrypted at hardware level on Nitro (verify)
Typical use	OS, app data, databases	Cache, scratch

7.4 EBS snapshots

1. A **snapshot** is a point-in-time, **incremental** backup stored in S3-backed AWS-managed storage. It is **Regional**, so it can be restored to **any AZ** in the Region (this is how you move a volume across AZs).
2. Restore by creating a **new volume** from the snapshot, attaching it and mounting it.
3. A volume restored from a snapshot loads blocks **lazily**, so first reads can be slow. **Fast Snapshot Restore (FSR)** or pre-reading the volume avoids this.
4. Automate with **Amazon Data Lifecycle Manager (DLM)** or **AWS Backup**: schedules, retention, cross-Region and cross-account copies.
5. A snapshot of a running volume is **crash-consistent**. For databases, freeze or flush, or use native backup, to get **application-consistent** data.
6. Snapshots can be encrypted. Encryption cannot be removed, and sharing encrypted snapshots needs a customer-managed key.
7. **Encryption by default** can be enabled per Region so that all new volumes are encrypted. **[MANDATORY]** for most organisations.

7.5 Resizing and modifying volumes (Elastic Volumes)

7.5.1 Simple definition

Elastic Volumes is an AWS EBS feature that lets you modify an EBS volume's size, volume type, IOPS, and throughput without detaching the volume or stopping the EC2 instance in supported configurations.

7.5.2 What can you modify?

- **Volume size:** Increase disk capacity, for example, 100 GB → 200 GB.
- **Volume type:** Change GP2 to GP3, for example.
- **IOPS:** Increase input/output operations per second when supported.
- **Throughput:** Increase data transfer performance when supported.

Example:

Before modification:

```
EC2 Instance
   └── EBS Volume
       Type: gp2
       Size: 100 GB
```

After modification:

```
EC2 Instance (running)
   └── EBS Volume
       Type: gp3
       Size: 200 GB
```

7.5.3 How to resize an EBS volume

Step 1: Check disk usage

```
lsblk
df -h
```

Step 2: Modify the volume in AWS Console

Go to **EC2 → Elastic Block Store → Volumes → Select volume → Actions → Modify volume**.

Step 3: Verify the new size

```
lsblk
df -h
```

The block device may show the new size before the filesystem does.

Step 4: Extend the partition/filesystem if required

Increasing the EBS volume size does not always automatically increase the usable filesystem space.

7.5.4 Linux commands to extend the filesystem

First, inspect the disk and filesystem:

```
lsblk
df -hT
```

For an XFS filesystem on a device mounted at `/data`:

```
sudo xfs_growfs /data
```

For an ext4 filesystem on a partition, after expanding the partition if necessary:

```
sudo resize2fs /dev/nvme1n1p1
```

If the partition itself needs to grow, use an appropriate partition-resizing procedure first. The correct commands depend on whether the EBS volume contains a partition, a whole-disk filesystem, or LVM.

> **Important:** Always verify the device name and take a suitable backup or snapshot before making storage changes.

7.5.5 Interview scenario

Question: Your production EC2 server has a 100 GB disk, and disk usage is reaching 95%. What will you do?

Answer:

1. Check disk and filesystem usage using `df -h` and `lsblk`.
2. Identify the EBS volume attached to the instance.
3. Take a snapshot if appropriate for recovery.
4. Modify the EBS volume from 100 GB to 200 GB.
5. Wait until the modification reaches a suitable state.
6. Extend the partition and filesystem if required.
7. Verify the available space using `df -h`.
8. Monitor the application and disk utilization.

7.5.6 Key interview points

- Elastic Volumes supports online modification for supported EBS configurations.
- Increasing an EBS volume's size does not necessarily extend the filesystem automatically.
- EBS volume modifications have service-specific limits and timing considerations.
- Increasing volume size or performance can increase costs.
- A snapshot provides a recovery point but does not replace proper change planning.

Memory trick: Modify EBS → Extend partition if needed → Extend filesystem → Verify.

7.6 Mounting a new volume safely

7.6.1 Simple definition

Mounting a new EBS volume means connecting an AWS virtual disk to a directory in Linux so that you can store and access data.

Example:

```
EC2 Instance
    |
    └── EBS Volume (100 GB)
            |
            └── /data
```

7.6.2 Step-by-step process

Step 1: Identify the new disk

```
lsblk
```

Check available disks and identify the newly attached volume.

```
sudo fdisk -l
```

> **Important:** Verify the disk carefully. Device names such as `/dev/nvme1n1` can vary between instances and may change after reboot.

Step 2: Check whether the disk contains data

```
sudo file -s /dev/nvme1n1
```

- If it reports a filesystem, do not format the disk.
- If it reports data or an existing partition, investigate before proceeding.
- Only format a confirmed new, empty volume.

> **Never run `mkfs` blindly. It can destroy existing data.**

Step 3: Create a filesystem (only for a confirmed empty disk)

For XFS:

```
sudo mkfs.xfs /dev/nvme1n1
```

For ext4:

```
sudo mkfs.ext4 /dev/nvme1n1
```

These commands assume you intend to create a filesystem directly on the whole disk, without a partition.

Step 4: Create a mount point

```
sudo mkdir -p /data
```

Step 5: Mount the volume

```
sudo mount /dev/nvme1n1 /data
```

Step 6: Verify the mount

```
df -hT
```

```
lsblk
```

```
findmnt /data
```

Confirm that the expected disk is mounted at `/data`.

Step 7: Make the mount persistent

Get the filesystem UUID:

```
sudo blkid /dev/nvme1n1
```

Example:

```
/dev/nvme1n1: UUID="abcd-1234" TYPE="xfs"
```

Edit `/etc/fstab`:

```
sudo vi /etc/fstab
```

Add the entry using your actual UUID:

```
UUID=abcd-1234 /data xfs defaults,nofail 0 2
```

Test the configuration:

```
sudo mount -a
```

Verify again:

```
findmnt /data
df -hT
```

Why use UUID? Device names can change, while a filesystem UUID provides a more reliable identifier.

7.6.3 Complete command flow

For a confirmed new, empty EBS volume with XFS:

```
# 1. Identify the disk
lsblk

# 2. Verify that the disk is empty
sudo file -s /dev/nvme1n1

# 3. Create the filesystem
sudo mkfs.xfs /dev/nvme1n1

# 4. Create the mount point
sudo mkdir -p /data

# 5. Mount the disk
sudo mount /dev/nvme1n1 /data

# 6. Verify
df -hT
findmnt /data

# 7. Get UUID
sudo blkid /dev/nvme1n1

# 8. Add the UUID entry to /etc/fstab
sudo vi /etc/fstab

# 9. Test the configuration
sudo mount -a
```

> **Do not rerun the formatting command on an existing volume.**

7.6.4 Production interview scenario

Question: You attached a new EBS volume to a production EC2 instance. How will you mount it safely?

Answer:

1. Identify the correct volume using `lsblk` and AWS console details.
2. Check for existing data and filesystems before formatting.

3. Create a filesystem only if the disk is confirmed empty.
4. Create a mount point and mount the volume.
5. Verify the filesystem and mount using `findmnt` and `df -hT`.
6. Add a UUID-based entry to `/etc/fstab`.
7. Test the configuration using `mount -a`.
8. Monitor the application and confirm that the mount persists across reboot.

Memory trick: Identify → Inspect → Format (only if empty) → Mount → Verify → Persist.

1. On Nitro instances, devices appear as `nvme*n1`. Names can change between boots, so **use UUIDs in `/etc/fstab`**.
2. A **wrong `/etc/fstab` entry can stop the instance from booting** correctly. Use `nofail` for non-root volumes and always run `mount -a` to test.

7.7 Troubleshooting commands

```
lsblk                # block devices and partitions
df -hT               # file system capacity and type
df -ih               # inode usage
sudo blkid           # file system identifiers (UUID)
findmnt              # mounted file systems
sudo du -xh / --max-depth=1 2>/dev/null | sort -h | tail   # largest directories
sudo lsof +L1        # deleted files still held open
iostat -xz 1 5       # per-device utilisation and latency (package sysstat)
```

Scenario: the disk is "100% full".

1. `df -hT` for space and `df -ih` for **inodes**. Millions of small files can exhaust inodes while space remains.
2. `du` to find large directories (logs, `/var/lib/docker`, temp files).
3. `lsof +L1`: a **deleted but still open** file (a log held by a running process) keeps using space until the process restarts.
4. Clean up or rotate logs, then grow the volume if the data is legitimate (section 7.5).

7.8 Common mistakes

1. Using `mkfs` on a volume that already holds data.
2. Not growing the partition and file system after the volume resize.
3. Putting important data on instance store.
4. Orphaned (unattached) volumes and old snapshots accumulating cost.
5. Assuming a snapshot is application-consistent.
6. `/etc/fstab` using device names, not UUIDs, or without `nofail`.

7.9 Interview pointers

Q6 (EBS vs instance store), Q11 (EBS types, snapshots), Q12 (resize), Q30 (disk full scenario).

---

8. Networking for EC2

8.1 IP addresses

Address	Description
Private IPv4	From the subnet range. Stays with the instance for its life (even across stop/start)
Public IPv4	Auto-assigned if the subnet or launch setting enables it. **Released on stop**, new one on start
Elastic IP (EIP)	Static public IPv4 you own until released. Can be moved between instances or ENIs. Charged like other public IPv4 addresses (verify pricing)
IPv6	Globally unique. Needs a dual-stack VPC and routes

A public IP alone does not make an instance reachable. You also need a route to an internet gateway and security group (and NACL) rules that allow the traffic.

8.2 Public vs private subnet

1. A subnet is **public** if its route table has a route (`0.0.0.0/0`) to an **internet gateway (IGW)**. Otherwise it is **private**.
2. Typical layout:

```
Internet -> IGW -> Public subnets: ALB, NAT gateway, (bastion if unavoidable)
                        |
                  Private subnets: application EC2 instances  (outbound via NAT / VPC endpoints)
                        |
                  Isolated subnets: databases (no internet route)
```

8.3 Internet gateway vs NAT gateway

Internet gateway	NAT gateway
Two-way internet access for resources with public IPs and routes	**Outbound-only** access to the internet for private-subnet instances
Used by public-subnet route tables	Lives in a public subnet. Private subnets route `0.0.0.0/0` to it
Horizontally scaled, no charge for the IGW itself	Billed per hour and per GB. Zonal, so use one per AZ for resilience

Use **VPC endpoints** (gateway endpoints for S3/DynamoDB, interface endpoints for SSM, ECR, Secrets Manager and others) to reach AWS services privately and cut NAT cost. For deeper VPC detail see `vpc.md`.

8.4 Elastic Network Interface (ENI)

1. An **ENI** has a primary private IP, optional secondary IPs, an optional public/Elastic IP, MAC address and **security groups**.
2. Every instance has a **primary ENI**. Additional ENIs (in the same AZ) can be attached for **management networks, dual-homed appliances or failover** (move the ENI to a standby instance to keep its IP).
3. The number of ENIs and IPs per instance depends on the instance type.
4. **Source/destination check** is on by default. Disable it only for NAT/router/firewall appliances.

8.5 Enhanced networking and placement groups

1. **ENA (Elastic Network Adapter)** gives high bandwidth and low latency on Nitro types. Keep the AMI's ENA driver current.
2. **Placement groups** influence where instances land:

Type	Behaviour	Use
Cluster	Instances close together in one AZ	Low latency, high throughput (HPC, tightly coupled). Higher risk of correlated failure
Spread	Each instance on distinct hardware (limit per AZ)	Small number of critical instances
Partition	Groups of instances on separate racks	Large distributed systems (Kafka, HDFS, Cassandra)

8.6 DNS and hostnames

1. Instances get private DNS names (and public DNS names if public IPs exist) if DNS support is enabled in the VPC.
2. For services, use **Route 53** records or a load balancer name, not instance IPs.

8.7 Interview pointers

Q7 (IP types), Q8 (public/private subnets, IGW vs NAT), Q20 (ENI and placement groups), Q32 (private instance cannot reach the internet).

---

9. Security Groups vs Network ACLs

9.1 Comparison

Feature	Security group (SG)	Network ACL (NACL)
Applies to	Network interfaces (instances, load balancers, endpoints)	Subnets
State	**Stateful**: return traffic is automatically allowed	**Stateless**: return traffic needs explicit rules

State	**Stateful**: return traffic is automatically allowed	**Stateless**: return traffic needs explicit rules
Rule types	**Allow only**	Allow and **deny**
Rule evaluation	All rules together (any allow wins)	In number order, first match wins
Default	Inbound denied, outbound allowed	Default NACL allows all
Typical use	Instance-level access control	Coarse subnet guardrails, blocking known bad ranges

9.2 Security Group Rules

AWS EC2 Security and Access — Simple Notes

Simple definition

A Security Group (SG) acts as a virtual firewall for EC2 instances. It controls inbound and outbound traffic.

- **Inbound rules:** Control traffic coming into the instance.
- **Outbound rules:** Control traffic leaving the instance.
- Security groups are stateful: response traffic is automatically allowed.
- Rules allow traffic; they do not have explicit deny rules.
- You can specify protocols, ports, and source or destination IP ranges or security groups.

Common rules

Purpose	Protocol	Port
SSH	TCP	22
HTTP	TCP	80
HTTPS	TCP	443
RDP	TCP	3389

Example

Suppose your EC2 server hosts a website.

```
Internet
   |
   | HTTPS : 443
   v
Security Group
   |
   v
EC2 Web Server
```

Allow HTTPS from the intended clients. For SSH, restrict access to a trusted IP or use Session Manager to avoid opening port 22.

Memory trick: Security Group = Stateful instance firewall.

9.3 NACL Gotchas

Simple definition

A Network Access Control List (NACL) is a firewall associated with a VPC subnet. It controls inbound and outbound traffic at the subnet level.

- NACLs are stateless: inbound and outbound traffic must be allowed separately.
- They support both allow and deny rules.
- Rules are evaluated in ascending rule-number order.
- The first matching rule decides whether traffic is allowed or denied.
- Each subnet is associated with one NACL at a time; a NACL can be associated with multiple subnets.
- The default NACL allows all inbound and outbound IPv4 traffic, and IPv6 traffic where configured. A custom NACL denies traffic by default until rules are added.

Important gotcha: Ephemeral ports

When a client connects to a server, the reply usually goes to a temporary client-side port.

Example:

```
Client
  |
  | Request to server port 443
  v
Server
  |
  | Reply to client's ephemeral port
  v
Client
```

If the NACL blocks the required return traffic, the connection may fail even when the Security Group is configured correctly.

Security Group vs NACL

Feature	Security Group	NACL
Applies to	Network interfaces / instances	Subnets
Stateful?	Yes	No
Allow rules	Yes	Yes
Deny rules	No explicit deny rules	Yes
Rule evaluation	All applicable rules	Lowest numbered matching rule first

Memory trick: SG = Stateful; NACL = Stateless.

---

10. EC2 Access: Key Pairs, Session Manager and Instance Connect

10.1 Key Pairs

A key pair is used to authenticate SSH access to Linux EC2 instances.

- **Public key:** Installed on the instance.
- **Private key:** Kept securely by you.
- Never share or expose your private key.
- The Security Group must allow inbound SSH on port 22 from your source IP when connecting directly over SSH.

Example:

```
        ssh -i my-key.pem ec2-user@<EC2-PUBLIC-IP>
```

The username depends on the AMI. For example, Amazon Linux commonly uses `ec2-user`, while Ubuntu commonly uses `ubuntu`.

10.2 SSH Troubleshooting Checklist

If you cannot connect to your EC2 instance, check these items in order:

1. **Instance status:** Is the instance running and passing status checks?
2. **Public IP:** Does it have a reachable public IP or DNS name if connecting over the internet?
3. **Security Group:** Is inbound TCP port 22 allowed from your current public IP?
4. **NACL:** Are inbound SSH and required outbound return/ephemeral traffic allowed?
5. **Route table:** Is there a valid route to the internet gateway for direct public access?
6. **Key pair:** Are you using the correct private key and file permissions?
7. **Username:** Are you using the correct AMI username?
8. **Network:** Is your local network, VPN, or corporate firewall blocking SSH?

9. **Instance OS:** Is the SSH service running and is the OS firewall allowing traffic?

Example key permission command:

```
chmod 400 my-key.pem
```

Common error meanings:

- `Connection timed out` — often a network route, Security Group, NACL, or firewall issue.
- `Permission denied (publickey)` — often the wrong key, username, or SSH configuration.
- `Connection refused` — the host may be reachable, but SSH may not be listening or may be blocked locally.

10.3 AWS Systems Manager Session Manager (Preferred)

Simple definition

Session Manager lets you access an EC2 instance through AWS Systems Manager without opening inbound SSH port 22 or managing SSH keys.

Requirements

- The instance must have a supported SSM Agent installed and running.
- The instance needs an appropriate IAM instance profile with permissions such as `AmazonSSMManagedInstanceCore`.
- It must have network connectivity to the Systems Manager service endpoints, through internet access or suitable VPC endpoints.
- Your IAM identity needs permission to start sessions.

How it works

```
Administrator
     |
     v
AWS Systems Manager
     |
     v
SSM Agent on EC2
     |
     v
Instance terminal
```

Benefits

- No inbound port 22 required.
- Centralized IAM access control.
- Session logging can be configured with CloudTrail and supported logging destinations.
- Useful for production administration and auditing.

Memory trick: Session Manager = Secure access without opening SSH.

10.4 EC2 Instance Connect

Simple definition

EC2 Instance Connect provides temporary SSH access by delivering a temporary public key to the instance.

1. The public key is pushed for approximately 60 seconds.
2. The instance must support EC2 Instance Connect and have the required software configured.
3. The Security Group and network must permit SSH on port 22 from the connection source.
4. The user still needs appropriate IAM permissions.

Example:

```
Administrator
     |
     v
EC2 Instance Connect
     |
     | Temporary public key
     v
EC2 Instance
     |
     v
SSH session
```

> **Important:** EC2 Instance Connect is not the same as Session Manager. Standard EC2 Instance Connect uses SSH and normally requires port 22 access. An EC2 Instance Connect Endpoint can provide private connectivity to instances without requiring a public IP, but the endpoint and security rules must be configured appropriately.

Memory trick: Instance Connect = Temporary SSH key.

10.5 Which Access Method Should You Choose?

Need	Recommended option
Production administration, auditing, and no inbound SSH port	Systems Manager Session Manager
Temporary SSH access with IAM control	EC2 Instance Connect
Automation tools that require SSH	Key pair or EC2 Instance Connect Endpoint, with tightly restricted Security Groups
Fleet-wide commands	Systems Manager Run Command
Patch management	Systems Manager Patch Manager

Interview answer

"For production EC2 access, I prefer Systems Manager Session Manager because it avoids opening inbound port 22 and supports centralized IAM control and session logging. For temporary SSH access, I can use EC2 Instance Connect. For automation tools that require SSH, I use tightly restricted network rules and secure key management."

10.6 Interview pointers

Q13 (SSH troubleshooting, key pairs), Q35 (SSM vs SSH), Q28 (cannot SSH).

---

11. User Data, cloud-init and the Instance Metadata Service

11.1 User data

1. **User data** is a script or cloud-init configuration passed at launch (limit about 16 KB before encoding). **cloud-init** runs it on most Linux AMIs.
2. By default it runs **once, at first boot, as root**. Behaviour can be changed (for example `#cloud-config` with `scripts_user: always` or `cloud_final_modules`).
3. A shell script must begin with `#!/bin/bash`. Output goes to `/var/log/cloud-init-output.log`.
4. Example for Amazon Linux 2023:

```
#!/bin/bash
set -euxo pipefail
dnf install -y nginx
systemctl enable --now nginx
echo "Hello from $(hostname -f)" > /usr/share/nginx/html/index.html
```

On other distributions use `yum` (older Amazon Linux/RHEL family) or `apt-get` (Ubuntu and Debian).

5. **User data is not secret.** It can be read through instance metadata and by identities allowed `ec2:DescribeInstanceAttribute`. **Never place passwords or keys in it.** Fetch secrets at boot from **Secrets Manager** or **Parameter Store** using the instance role.
6. Long installs delay readiness. Bake heavy software into an AMI and keep user data short.
7. If an Auto Scaling health check starts before user data finishes, set a sensible **grace period** (see Auto Scaling notes).

11.2 Troubleshooting user data

```
sudo cloud-init status --long            # status: done / running / error, with details
sudo less /var/log/cloud-init.log        # cloud-init engine log
sudo less /var/log/cloud-init-output.log # stdout/stderr of your script
sudo systemctl status nginx              # did the service start?
curl -s http://localhost/                # does the app answer locally?
```

Common causes: missing `#!`, a package manager mismatch, no outbound internet (no NAT or endpoint) so installs fail, a script error halting early, or code that assumes interactive input.

11.3 Instance Metadata Service (IMDS)

1. The IMDS is a local HTTP service at **`http://169.254.169.254`**. It returns instance ID, AZ, AMI ID, network details, user data and **temporary role credentials**.
2. **IMDSv1** answers simple GET requests. It is vulnerable to **SSRF** (server-side request forgery) attacks where a vulnerable web app is tricked into fetching the metadata URL and leaking role credentials.
3. **IMDSv2** is session-oriented: you first `PUT` to get a **token**, then send it in a header. This blocks most SSRF attacks.

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
        -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/iam/security-credentials/
# the last call lists the ROLE NAME only. Do not print the credentials in shared logs.
```

4. **[MANDATORY]** Require IMDSv2: `HttpTokens=required`. Set it in the launch template, and consider an **account-level default** and an **SCP/IAM condition** (`ec2:MetadataHttpTokens`) to enforce it.
5. **Hop limit:** the default `HttpPutResponseHopLimit` is 1, which means **containers on the instance (an extra network hop) cannot reach IMDS**. Set it to 2 only where containers genuinely need the instance role. Prefer pod or task roles (IRSA, EKS Pod Identity, ECS task roles) so pods do not use the node role.
6. Disable IMDS entirely (`HttpEndpoint=disabled`) for instances that do not need it.

11.4 Interview pointers

Q14 (user data and cloud-init), Q15 (IMDSv2), Q36 (SSRF credential theft).

---

12. IAM Roles and Instance Profiles

12.1 The problem

Applications on EC2 need AWS permissions (read an S3 bucket, write logs, fetch a secret). **Do not store access keys on the**

instance. Use an **IAM role**.

12.2 How it works

1. Create an **IAM role** whose **trust policy** allows `ec2.amazonaws.com` to assume it.
2. Attach a **least-privilege permissions policy** (specific actions and resources).
3. The role is attached to the instance through an **instance profile** (the console creates it with the role automatically. The CLI and Terraform need it explicitly).
4. The instance gets **temporary credentials** from IMDS, rotated automatically before expiry. The AWS SDK and CLI find them through the **default credential provider chain**.
5. An instance can have **one instance profile** (one role) at a time. You can replace it on a running instance (`associate-iam-instance-profile`).
6. To attach a role at launch, the caller needs `iam:PassRole` for that role. Restrict this permission.

```
App (AWS SDK) -> IMDS (IMDSv2) -> temporary credentials (access key, secret, session token)
             -> signed API call -> IAM evaluates the ROLE's policies -> allow or deny
CloudTrail records the call with the role session name (instance ID)
```

12.3 Example: read one S3 bucket

Trust policy:

```
        {
          "Version": "2012-10-17",
          "Statement": [{
            "Effect": "Allow",
            "Principal": { "Service": "ec2.amazonaws.com" },
            "Action": "sts:AssumeRole"
          }]
        }
```

Permissions policy (least privilege):

```
        {
          "Version": "2012-10-17",
          "Statement": [
            { "Effect": "Allow", "Action": "s3:ListBucket",
              "Resource": "arn:aws:s3:::<bucket-name>" },
            { "Effect": "Allow", "Action": ["s3:GetObject"],
              "Resource": "arn:aws:s3:::<bucket-name>/reports/*" }
          ]
        }
```

Check the identity in use (safe to run, shows no secret):

```
        aws sts get-caller-identity
```

Expected output (shape):

```
        { "UserId": "AROA...:i-0abc123...", "Account": "<account-id>", "Arn": "arn:aws:sts::<account-id>:assumed-role/<role-name>/i-0abc123..." }
```

An `assumed-role/<role-name>/i-...` ARN means the instance role is being used. If you see an `user/...` ARN, **environment variables or a config file with static keys override the role** (a common bug).

12.4 Best practices

1. One role per workload, **least privilege**, resource-level ARNs and conditions.
2. Add SSM access (`AmazonSSMManagedInstanceCore`) only where Session Manager is used.
3. Use **permission boundaries** or **SCPs** to cap what roles can do.

4. Monitor with **CloudTrail** and **IAM Access Analyzer**. Remove unused permissions.
5. Protect IMDS (IMDSv2) because role credentials live there.
6. Do not give administrator permissions just to "make it work".

12.5 Interview pointers

Q10 (IAM role vs key pair), Q33 (AccessDenied from an instance), Q34 (hardening), Q36 (credential theft).

---

13. Launch Templates and Auto Scaling (Summary)

13.1 Launch templates

1. A **launch template** stores launch settings: AMI, instance type, key pair (optional), security groups, instance profile, EBS mappings, user data, network interfaces, **metadata options**, market options (Spot) and tags.
2. Templates are **versioned**. Pin a version in an ASG, test a new version, roll out with **instance refresh**, and roll back by selecting the older version.
3. They replace **launch configurations**, which are deprecated.
4. Set safe defaults in the template: **IMDSv2 required, encrypted EBS, no public IP in private subnets, instance profile with least privilege, detailed monitoring if needed**.

13.2 Auto Scaling groups (short version)

Setting	Meaning
Minimum	Floor
Desired	Current target count
Maximum	Ceiling and cost safety limit
Launch template	How new instances are built
Health checks	EC2 status and (when enabled) load balancer health
Scaling policies	Target tracking, step, simple, scheduled and predictive

Example: min 2, desired 2, max 6. A scaling policy can raise desired up to 6 and bring it back down.

1. Use **multiple AZs**, **ELB health checks** behind a load balancer and a **grace period** that matches boot time.
2. Prefer **target tracking** on a metric that reflects load (for example `ALBRequestCountPerTarget`).
3. Full coverage (lifecycle hooks, instance refresh, warm pools, Spot mixes, troubleshooting) is in **`ec2-auto-scaling.md`**.

13.3 Interview pointers

Q16 (launch templates).

---

14. Monitoring, Status Checks and Recovery

14.1 CloudWatch metrics for EC2

1. **Basic monitoring:** metrics every **5 minutes** (free). **Detailed monitoring:** every **1 minute** (extra cost).
2. Default hypervisor-level metrics: `CPUUtilization`, `NetworkIn/Out`, `DiskReadOps/WriteOps` (instance store), `StatusCheckFailed*`, and for T types `CPUCreditBalance`.
3. **Memory and disk space are not collected by default.** Install the **CloudWatch agent** to publish memory, disk, swap and custom metrics and logs.
4. EBS volume metrics (in the EBS namespace) include `VolumeQueueLength`, `VolumeReadOps/WriteOps`, `VolumeIdleTime` and `BurstBalance` (gp2).

14.2 Status checks

Check	What it tests	Typical causes of failure	Typical fix
System	The AWS side: host power, network	Host hardware or networking problem	**Stop and start** (moves to a new

status check	connectivity to the host, hardware		host) or let **auto recovery** do it
Instance status check	The guest OS and its network config	Kernel panic, corrupt file system or bad `fstab`, exhausted memory, wrong network configuration	Reboot, read the console output, fix with a rescue instance if needed
Attached EBS status check	Volume reachability and I/O	Volume impaired	Detach and re-attach, restore from snapshot

14.3 Automatic recovery

1. **Simplified automatic recovery** is on by default for supported instance types. When the system status check fails, EC2 recovers the instance onto new hardware, keeping the instance ID, private IP, EIP, metadata and volumes. **Memory contents are lost** (it reboots).
2. You can also create a CloudWatch alarm on `StatusCheckFailed_System` with the **recover** action. For ASG members the group replaces the instance instead.
3. Recovery does not apply to instances with instance store volumes (verify) and does not fix OS-level (instance check) problems.

14.4 Diagnostic tools

```
aws ec2 describe-instance-status --instance-ids <id> --include-all-instances
aws ec2 get-console-output --instance-id <id> --latest --output text | tail -n 50
aws ec2 get-console-screenshot --instance-id <id>     # image of the console (Nitro)
```

1. **EC2 Serial Console** gives text access to the console (boot problems, broken network config) without a network connection. Enable it per account and control with IAM.
2. **CloudTrail** shows who stopped, terminated or changed an instance. **AWS Config** tracks configuration history. **EventBridge** receives instance state-change and scheduled-event notifications.
3. **Scheduled events** (retirement, maintenance) appear in `describe-instance-status`. Act before the date by stop/start (for retirement) or let it reboot in the window.

14.5 Recommended alarms

Alarm	Example	Why
`StatusCheckFailed_System`	>= 1 for 2 minutes -> recover action	Host problems
`StatusCheckFailed_Instance`	>= 1 for 5 minutes -> notify	OS problems
`CPUUtilization`	> 80% for 10 minutes	Saturation
`CPUCreditBalance` (T types)	Near zero	Throttling coming
Memory used % (agent)	> 85%	OOM risk
Disk used % and inodes (agent)	> 80%	Full disk
`NetworkOut` anomaly	Unusual spike	Data exfiltration or abuse
Auto Scaling or ALB unhealthy host count	> 0	Service impact

14.6 Interview pointers

Q18 (status checks and recovery), Q31 (status check failed scenario), Q29 (high CPU).

---

15. Hands-On: CLI and Terraform

> Use a **sandbox account**. Running instances cost money. Replace every `<placeholder>`. No secrets appear below.

15.1 AWS CLI: launch and inspect

```
export AWS_REGION=<region>

# Find the latest Amazon Linux 2023 AMI through a public SSM parameter (no hard-coded AMI ID)
AMI_ID=$(aws ssm get-parameter \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query 'Parameter.Value' --output text)
echo "$AMI_ID"

# Launch one private instance: IMDSv2 required, encrypted gp3 root, no key pair (use Session Manager)
aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type t3.micro \
  --subnet-id <private-subnet-id> \
  --security-group-ids <sg-id> \
  --iam-instance-profile Name=<instance-profile-name> \
  --metadata-options HttpTokens=required,HttpEndpoint=enabled,HttpPutResponseHopLimit=1 \
  --block-device-mappings 'DeviceName=/dev/xvda,Ebs={VolumeSize=20,VolumeType=gp3,Encrypted=true,DeleteOnTermination=true}' \
  --monitoring Enabled=true \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=lab-web-01},{Key=Environment,Value=lab}]' \
                       'ResourceType=volume,Tags=[{Key=Name,Value=lab-web-01-root}]'
```

Explanation

1. The SSM parameter always resolves to the current AMI, so scripts do not rot.
2. `HttpTokens=required` enforces IMDSv2. Hop limit 1 blocks containers from using the node role.
3. No `--key-name`: access is through Session Manager.
4. `t3.micro` is for labs.

Expected output (shape): JSON with `"State": {"Name": "pending"}` and an `InstanceId` such as `i-0abc...`.

```
aws ec2 wait instance-running --instance-ids <instance-id>

aws ec2 describe-instances --instance-ids <instance-id> \
  --query 'Reservations[].Instances[].
{Id:InstanceId,State:State.Name,Type:InstanceType,AZ:Placement.AvailabilityZone,PrivateIp:PrivateIpAddress,PublicIp:PublicIpAddress
,IMDS:MetadataOptions.HttpTokens,Profile:IamInstanceProfile.Arn}' \
  --output table
```

Expected output (shape):

```
-------------------------------------------------------------------------------
|                            DescribeInstances                                |
+-----+------------------+---------+-----------+------------+---------------+
| AZ  | Id               | IMDS    | PrivateIp | State      | Type          |
+-----+------------------+---------+-----------+------------+---------------+
| ... | i-0abc...        | required| 10.0.x.x  | running    | t3.micro      |
```

15.2 AWS CLI: day-two operations

```
# Lists with filters (running instances for one tag)
aws ec2 describe-instances --filters Name=tag:Environment,Values=lab Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType,PrivateIpAddress]' --output table

# Stop, change type, start (short outage)
aws ec2 stop-instances --instance-ids <id> && aws ec2 wait instance-stopped --instance-ids <id>
aws ec2 modify-instance-attribute --instance-id <id> --instance-type "{\"Value\": \"m7i.large\"}"
aws ec2 start-instances --instance-ids <id>

# Turn on termination protection
aws ec2 modify-instance-attribute --instance-id <id> --disable-api-termination

# Require IMDSv2 on an existing instance
aws ec2 modify-instance-metadata-options --instance-id <id> \
  --http-tokens required --http-endpoint enabled

# Account-level default for NEW instances in this Region
aws ec2 modify-instance-metadata-defaults --http-tokens required --http-put-response-hop-limit 1

# Turn on encryption by default for new EBS volumes in this Region
aws ec2 enable-ebs-encryption-by-default

# Create an AMI (reboots by default for consistency; add --no-reboot only if crash-consistent is acceptable)
aws ec2 create-image --instance-id <id> --name "web-$(date +%Y%m%d%H%M)" \
  --tag-specifications 'ResourceType=image,Tags=[{Key=Name,Value=web-golden}]'
aws ec2 wait image-available --image-ids <ami-id>

# Copy an AMI to another Region (run against the DESTINATION Region)
aws ec2 copy-image --region <dr-region> --source-region <source-region> \
  --source-image-id <ami-id> --name web-golden-dr --encrypted

# Clean up an AMI: deregister, then delete its snapshots
aws ec2 deregister-image --image-id <ami-id>
aws ec2 delete-snapshot --snapshot-id <snapshot-id>

# Terminate (permanent)
aws ec2 terminate-instances --instance-ids <id>
```

15.3 AWS CLI: EBS operations

```
# New volume in the same AZ as the instance, encrypted gp3
aws ec2 create-volume --availability-zone <az> --size 50 --volume-type gp3 --encrypted \
  --tag-specifications 'ResourceType=volume,Tags=[{Key=Name,Value=lab-data}]'
aws ec2 attach-volume --volume-id <vol-id> --instance-id <id> --device /dev/sdf

# Grow a volume online (then grow partition and file system inside the OS, section 7.5)
aws ec2 modify-volume --volume-id <vol-id> --size 100
aws ec2 describe-volumes-modifications --volume-ids <vol-id> \
  --query 'VolumesModifications[0].[ModificationState,Progress]'

# Snapshot, then restore into a DIFFERENT AZ (snapshots are Regional)
aws ec2 create-snapshot --volume-id <vol-id> --description "pre-change $(date +%F)" \
  --tag-specifications 'ResourceType=snapshot,Tags=[{Key=Name,Value=lab-data-snap}]'
aws ec2 wait snapshot-completed --snapshot-ids <snap-id>
aws ec2 create-volume --snapshot-id <snap-id> --availability-zone <other-az> --volume-type gp3

# Find unattached (orphaned) volumes, a common cost leak
aws ec2 describe-volumes --filters Name=status,Values=available \
  --query 'Volumes[].[VolumeId,Size,VolumeType,CreateTime]' --output table
```

15.4 AWS CLI: investigation toolbox

```
        # Status checks (system vs instance) and scheduled events
        aws ec2 describe-instance-status --instance-ids <id> --include-all-instances

        # Console output (boot messages, kernel panic, cloud-init)
        aws ec2 get-console-output --instance-id <id> --latest --output text | tail -n 60

        # Is the instance reachable by Session Manager?
        aws ssm describe-instance-information \
          --query 'InstanceInformationList[].[InstanceId,PingStatus,PlatformName,AgentVersion]' --output table

        # Security group rules and the route table used by the instance's subnet
        aws ec2 describe-security-groups --group-ids <sg-id> --query 'SecurityGroups[0].[IpPermissions,IpPermissionsEgress]'
        aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=<subnet-id> \
          --query 'RouteTables[].Routes'

        # Who stopped or changed this instance? (CloudTrail)
        aws cloudtrail lookup-events --lookup-attributes AttributeKey=ResourceName,AttributeValue=<instance-id> \
          --max-results 20 --query 'Events[].[EventTime,EventName,Username]' --output table

        # Which identity is the instance using? (run ON the instance, via SSM)
        aws sts get-caller-identity

        # Instance type facts
        aws ec2 describe-instance-types --instance-types m7i.large \
          --query 'InstanceTypes[0].
{vCPU:VCpuInfo.DefaultVCpus,MemMiB:MemoryInfo.SizeInMiB,Net:NetworkInfo.NetworkPerformance,EbsBw:EbsInfo.EbsOptimizedInfo.BaselineB
andwidthInMbps}'
```

Reachability tests from inside a VPC host:

```
        nc -zv <target-host> <port>
        curl -sv --max-time 5 https://aws.amazon.com -o /dev/null      # outbound internet (needs NAT or IGW path)
        nslookup <name>
```

15.5 Linux commands on the instance (first five minutes)

```
        uptime                       # load average
        top -o %CPU                  # or: htop. Who uses CPU?  (shift+M sorts by memory)
        free -h                      # memory and swap
        vmstat 1 5                   # r (run queue), si/so (swap), wa (iowait)
        iostat -xz 1 5               # disk utilisation and await
        df -hT; df -ih               # space and inodes
        ss -tulpn                    # listening ports and owning processes
        journalctl -p err -b --no-pager | tail -n 50     # errors since boot
        dmesg -T | tail -n 30        # kernel messages (OOM killer, disk errors)
        systemctl --failed           # failed services
```

Look for: high `wa` (disk wait) vs high `us` / `sy` (CPU work), `Out of memory: Killed process` in `dmesg`, and full disks or inodes.

---

16. Production Troubleshooting Scenarios

Each scenario follows: **Problem → Symptoms/impact → Possible causes → Investigation → Procedure → Root cause → Resolution → Prevention → How to explain it in an interview.**

16.1 Beginner scenarios

A1. Cannot SSH to an EC2 instance

1. **Problem:** `ssh` times out, is refused, or says permission denied.
2. **Symptoms and impact:** No admin access. The app may still be serving traffic.
3. **Possible causes:**
    1. **Timeout:** security group lacks TCP 22 from your IP (your IP changed), NACL blocks, no public IP or no IGW route, instance stopped or failing status checks, a corporate firewall blocks 22.
    2. **Connection refused:** `sshd` not running or listening on another port, host firewall.
    3. **Permission denied (publickey):** wrong username, wrong key, key file permissions too open, `authorized_keys` modified or wrong permissions.
    4. Disk full or out of memory so `sshd` cannot work.
4. **Tools:** `ssh -vvv`, `describe-instance-status`, `describe-security-groups`, `describe-route-tables`, `get-console-output`, Session Manager.
5. **Procedure:**
    1. Instance `running` with passing status checks?
    2. Public IP present and the subnet route table has `0.0.0.0/0` to an IGW (or use SSM for private instances)?
    3. SG inbound 22 from your **current** IP? NACL inbound 22 and outbound ephemeral ports?
    4. Right user (`ec2-user`, `ubuntu`) and `chmod 400` on the key?
    5. If you can still use Session Manager, check `systemctl status sshd`, `df -h`, `/var/log/secure`.
6. **Root cause (typical):** The SG allowed only the office IP and the engineer was on a different network, or the wrong username was used.
7. **Resolution:** Fix the SG rule (temporary, narrow, time-boxed), fix the username or key. **Better:** move to Session Manager and remove inbound 22.
8. **Prevention:** Session Manager as the standard, no public SSH, key pair rotation if SSH is needed, IaC for SG rules.
9. **Interview explanation:** "I separate network from authentication. A timeout means the network path, SG, NACL or route. 'Permission denied' means the network worked and the key or username is wrong. I prefer Session Manager so there is no SSH exposure."

A2. Disk is 100% full

1. **Problem:** Applications fail with `No space left on device`.
2. **Symptoms and impact:** Failed writes, crashed services, possibly broken SSH and package installs.
3. **Possible causes:** Growing logs, core dumps, a large temp file, Docker images and layers, **inode exhaustion** (millions of tiny files), **deleted files still held open** by a process, an undersized volume.
4. **Tools:** `df -hT`, `df -ih`, `du`, `lsof +L1`, CloudWatch agent disk metrics.
5. **Procedure:**
    1. `df -hT` (space) and `df -ih` (inodes). Which one is at 100%?
    2. `sudo du -xh / --max-depth=1 | sort -h | tail` and drill down.
    3. `sudo lsof +L1`: deleted but open files. Restart or signal the owning process to release space.
    4. Free space safely (rotate or compress logs, prune Docker, remove temp files). **Do not delete application data blindly.**
    5. If the data is legitimate, grow the EBS volume and file system (section 7.5).
6. **Root cause (typical):** An application log without rotation, or a log deleted while the process kept writing to it.
7. **Resolution:** Clean up, restart the process holding deleted files, configure `logrotate` and journald limits, and resize if needed.
8. **Prevention:** CloudWatch agent alarms at 80% for space and inodes, log shipping to CloudWatch Logs with local rotation, separate data volumes, capacity reviews.
9. **Interview explanation:** "I check space and inodes, find the biggest consumer, look for deleted-but-open files, free space safely, then fix the cause with rotation and alarms. Expanding the volume is the last step."

A3. Instance in a private subnet cannot reach the internet or Session Manager

1. **Problem:** `yum` / `dnf` times out, `curl` to the internet hangs, the instance does not appear in Session Manager.
2. **Symptoms and impact:** User data installs fail, no patches, no SSM access.
3. **Possible causes:** No **NAT gateway** (or it sits in a subnet without an IGW route), the **private route table has no `0.0.0.0/0` to NAT**, NAT in a different AZ that failed, **security group outbound** too tight, **NACL** blocking, missing **VPC endpoints** for SSM (`ssm`, `ssmmessages`, `ec2messages`) and the endpoint SG not allowing 443 from the instance, instance role missing `AmazonSSMManagedInstanceCore`, DNS support off in the VPC.
4. **Tools:** route tables, `nc -zv`, `curl -v`, VPC Reachability Analyzer, flow logs, `describe-instance-information`.
5. **Procedure:**
    1. Route table of the instance's subnet: where does `0.0.0.0/0` point? Is the NAT gateway `available` and in a **public** subnet with an IGW route?

    2. SG outbound 443, NACL rules both ways.
    3. For SSM: endpoints exist (or NAT path), endpoint SG allows 443 from the instance SG, **Private DNS** enabled on interface endpoints, instance profile attached.
    4. Use Reachability Analyzer from the instance ENI to the endpoint or an internet destination.
6. **Root cause (typical):** A new private subnet was created with the default route table (local route only).
7. **Resolution:** Associate the correct route table (NAT) or add VPC endpoints. Fix SG and role.
8. **Prevention:** IaC modules for subnets and routes, endpoints for SSM and S3, a smoke test at launch.
9. **Interview explanation:** "A private instance needs an outbound path: NAT or endpoints. I follow the packet: instance SG, NACL, route table, NAT or endpoint, then DNS. SSM needs the role, the agent and a network path."

16.2 Intermediate scenarios

B1. Status check failed: system vs instance

1. **Problem:** CloudWatch or the console shows `1/2 checks failed` or `0/2 checks passed`.
2. **Symptoms and impact:** The instance is unreachable or degraded. The load balancer marks it unhealthy.
3. **Possible causes:**
    1. **System status failed:** AWS host or network problem.
    2. **Instance status failed:** kernel panic, corrupt or full root file system, bad `/etc/fstab`, out-of-memory, wrong network configuration, a failed boot.
4. **Tools:** `describe-instance-status`, `get-console-output`, console screenshot, EC2 Serial Console, CloudTrail (who changed what), recent changes.
5. **Procedure:**
    1. Which check failed?
    2. **System check:** stop and start the instance (moves it to new hardware) or rely on auto recovery. Instances in an ASG are replaced.
    3. **Instance check:** read the console output for panic, fsck or fstab errors. Try a reboot. If it still fails, **stop** the instance, **detach the root volume**, attach it to a rescue instance in the same AZ, fix the cause (fstab, full disk, network config), reattach and start.
    4. Snapshot the volume before repairs.
6. **Root cause (typical):** A bad `fstab` entry for a missing data volume, or a full root disk after log growth.
7. **Resolution:** Repair through a rescue instance or the serial console, restore from a snapshot/AMI if the file system is damaged.
8. **Prevention:** `nofail` and UUIDs in `fstab`, alarms for disk and memory, auto-recovery and ASG replacement, immutable instances rebuilt from AMIs.
9. **Interview explanation:** "System check is AWS's side, so stop and start or auto recovery. Instance check is my OS, so I read the console output and, if needed, repair the root volume on a rescue instance."

B2. High CPU on an instance (burstable credit exhaustion)

1. **Problem:** The application is slow and CPU is high, or a T-family instance suddenly performs badly.
2. **Symptoms and impact:** Latency, timeouts and queue growth. On T types, CPU is capped at baseline after credits run out.
3. **Possible causes:** A runaway process, a new release with expensive code, a traffic spike, a crypto-miner after compromise, heavy garbage collection, **CPU credits exhausted**, `iowait` misread as CPU, noisy neighbour (rare on Nitro).
4. **Tools:** `CPUUtilization`, `CPUCreditBalance`, `top` / `htop`, `vmstat`, `pidstat`, application profiling, GuardDuty.
5. **Procedure:**
    1. When did it start, and what changed?
    2. T type? Check `CPUCreditBalance` and `CPUSurplusCreditBalance`. A balance at zero with high utilisation at baseline percent explains it.
    3. `top`: which process? User, system or iowait time? Unknown processes need a security review.
    4. Check application logs and slow endpoints.
6. **Root cause (typical):** A steady workload on a `t3.medium` drained credits during a promotion.
7. **Resolution:** Fix the code or process, scale out (Auto Scaling), move to a non-burstable type (M or C) or enable unlimited mode (watch the cost).
8. **Prevention:** Right-size from metrics (Compute Optimizer), alarms on credits, load tests before releases, avoid T types for steady load.
9. **Interview explanation:** "I check whether it is CPU, iowait or throttling. For T types I look at credits first. Then I find the process and decide to fix, scale out or change instance family."

B3. AccessDenied when the application calls AWS

1. **Problem:** `AccessDenied` or `UnauthorizedOperation`, or `Unable to locate credentials`.
2. **Symptoms and impact:** The application cannot read S3, fetch secrets or write logs.
3. **Possible causes:** No instance profile attached, the role lacks the permission, a **resource policy or SCP** denies, wrong resource ARN or Region, **static keys in environment variables override the role**, IMDSv2 hop limit 1 in a **container** (cannot reach IMDS), a KMS key policy missing the role, expired session after a long-running process cached credentials badly.
4. **Tools:** `aws sts get-caller-identity` (run as the app), CloudTrail (the denial and its reason), IAM Policy Simulator, IAM Access Analyzer.
5. **Procedure:**
    1. `aws sts get-caller-identity` on the instance (or inside the container). Which principal appears? Expect `assumed-role/<role>/<instance-id>`.
    2. Attach or fix the profile if missing.
    3. Read the CloudTrail event for the exact action and resource and the error message.
    4. Check identity policy, permission boundary, SCP, resource policy (bucket policy, KMS key policy).
    5. In containers, check the hop limit or use task or pod roles.
6. **Root cause (typical):** An old `AWS_ACCESS_KEY_ID` in the service environment overrode the instance role, or the role lacked `kms:Decrypt` on the bucket's key.
7. **Resolution:** Remove static keys, add the specific missing permission (not `*`), fix the policy chain.
8. **Prevention:** No long-lived keys on instances, IAM least privilege with tests in CI, Access Analyzer, per-workload roles.
9. **Interview explanation:** "I first prove which identity the code uses with `get-caller-identity`, then read the CloudTrail denial to see what was missing, and fix the narrowest permission."

B4. User data did not run or the application is not up after launch

1. **Problem:** The instance is `running` but nginx or the app is missing, or the load balancer reports it unhealthy.
2. **Symptoms and impact:** New instances never serve traffic. Auto Scaling loops.
3. **Possible causes:** Missing `#!`, syntax error, package manager mismatch (`apt` vs `dnf`), **no outbound internet** for installs, an IAM or Secrets Manager call failing, the script waits for input, runs before dependencies, user data changed but the instance was not replaced (it runs only at first boot).
4. **Tools:** `cloud-init status --long`, `/var/log/cloud-init-output.log`, `get-console-output`, `systemctl status`, `curl localhost`.
5. **Procedure:**
    1. `sudo cloud-init status --long` for `done` or `error`.
    2. Read `/var/log/cloud-init-output.log` for the failing line.
    3. Test the network path (A3) and the package source.
    4. Run the commands by hand to reproduce.
    5. Fix the script with `set -euxo pipefail`, or move installs into the AMI.
6. **Root cause (typical):** A private subnet without NAT, so `dnf install` timed out.
7. **Resolution:** Add NAT or endpoints, or bake the software into the AMI, then replace the instance.
8. **Prevention:** Golden AMIs, short user data, tested launch in CI, health check grace period aligned with boot time.
9. **Interview explanation:** "User data runs once at first boot. I read the cloud-init logs for the failing step, which is usually network or packages, and I reduce the dependency by baking an AMI."

16.3 Advanced scenarios

C1. Lost key pair or broken access: recover with a rescue instance

1. **Problem:** No SSH key, Session Manager unavailable, or SSH is broken by a bad `sshd_config` or `fstab`.
2. **Symptoms and impact:** Locked out of a production instance that must be preserved.
3. **Possible causes:** The key owner left, the key was deleted, `authorized_keys` overwritten, a bad config edit, SSM agent not installed.
4. **Tools:** EC2 Serial Console, EC2 Instance Connect, a **rescue instance**, snapshots, `modify-instance-attribute`.
5. **Procedure:**
    1. **Try the safe paths first:** Session Manager (if the agent and role exist), EC2 Instance Connect, Serial Console.
    2. Snapshot the root volume (safety net).
    3. **Stop** the target instance (not terminate). Detach its **root volume**.
    4. Attach it as a secondary volume to a **rescue instance in the same AZ**. Mount it (`lsblk`, `mount /dev/<dev>1 /mnt/rescue`, check `blkid` for duplicate UUIDs with XFS using `-o nouuid`).
    5. Fix the file: add a new public key to `/mnt/rescue/home/ec2-user/.ssh/authorized_keys` (correct owner and mode 600) or repair `sshd_config` / `fstab`.
    6. Unmount, detach, reattach as the **root device** (`/dev/xvda` or the original name) and start.
6. **Root cause (typical):** The only key holder left the company and no SSM access was set up.

7. **Resolution:** Regain access, then install the SSM agent and role, and remove long-lived key dependence.
8. **Prevention:** Session Manager everywhere, key pairs managed in a team vault if needed, immutable infrastructure (replace instead of repair), IaC.
9. **Interview explanation:** "I snapshot first, try SSM or serial console, and if those fail, I swap the root volume to a rescue instance, repair the file and put it back. Then I fix the process gap so it cannot happen again."

C2. InsufficientInstanceCapacity or launch failures

1. **Problem:** `run-instances` or an Auto Scaling launch fails.
2. **Symptoms and impact:** Cannot scale out or replace instances during a peak.
3. **Common errors and causes:**
    1. `InsufficientInstanceCapacity`: AWS lacks that type in that AZ now.
    2. `VcpuLimitExceeded` / `InstanceLimitExceeded`: Service Quotas limit reached.
    3. `InvalidParameterValue` or `InvalidAMIID.NotFound`: wrong AMI for the Region or architecture (Arm vs x86).
    4. `Client.InternalError` with encrypted volumes: key policy denies the service role.
    5. `InsufficientFreeAddressesInSubnet`: subnet out of IPs.
    6. `UnauthorizedOperation` or `iam:PassRole` missing.
4. **Procedure:** read the exact error, check quotas (Service Quotas), subnet free IPs, AMI Region and architecture, KMS key policy and IAM.
5. **Resolution:** Try another instance type or AZ (mixed instances policy), request a quota increase, add subnets, fix key policy or role, use **On-Demand Capacity Reservations** for critical capacity.
6. **Prevention:** Diversify types and AZs, alarms on quota usage, capacity reservations for DR and key events, test launches in the DR Region regularly.
7. **Interview explanation:** "I read the error code. Capacity is solved with flexibility across types and AZs, quota with Service Quotas, and key or IAM errors with the right policy. For critical capacity I reserve it."

C3. A compromised instance: credential theft via SSRF or an exposed port

1. **Problem:** GuardDuty or billing shows crypto-mining, unusual API calls from the instance role, or outbound traffic to suspicious hosts.
2. **Symptoms and impact:** Possible data loss, cost spike and a security incident.
3. **Possible causes:** An app vulnerable to **SSRF** with **IMDSv1** enabled, exposed SSH/RDP with weak access, an unpatched application, leaked keys, a malicious or outdated AMI, an over-privileged role.
4. **Tools:** GuardDuty, CloudTrail (calls made with the role session), VPC flow logs, SSM, EBS snapshots, AWS Config.
5. **Procedure:**
    1. **Contain:** replace the instance's security group with a **quarantine SG** (no inbound and no outbound), leaving it running to preserve memory evidence, or follow your incident policy. Do not simply terminate.
    2. **Revoke the role's credentials:** in IAM use **Revoke active sessions** on the role (it adds a deny for sessions issued before now). The instance then needs a new role session for legitimate use after the fix.
    3. **Preserve evidence:** snapshot volumes and capture memory if your process requires it, and export logs.
    4. **Investigate:** CloudTrail for what the role did, flow logs for traffic, system logs. Rotate any secrets the role could read.
    5. **Eradicate and recover:** build a **clean instance from a trusted AMI**, patch the vulnerability, restore data from a known-good backup.
6. **Root cause (typical):** A web app with SSRF and IMDSv1 let an attacker fetch role credentials.
7. **Resolution:** Contain, revoke, rebuild, rotate secrets, and notify per policy.
8. **Prevention: IMDSv2 required** account-wide, least-privilege roles, patching, WAF, GuardDuty and Inspector, no public admin ports, SCP enforcing IMDSv2.
9. **Interview explanation:** "I isolate with a quarantine security group, revoke the role sessions, keep evidence, find out what the credentials did through CloudTrail, then rebuild from a clean image. The structural fix is IMDSv2 and least privilege."

---

17. Security, Reliability and Best Practices

17.1 Mandatory controls vs optional design choices

Area	Mandatory	Optional (depends on need and cost)
Network	Private subnets for app tiers, SGs reference SGs, no `0.0.0.0/0` on admin or DB ports	VPC endpoints, NACL guardrails, Network Firewall
Access	Session Manager or controlled SSH, no shared keys	EC2 Instance Connect, session logging to S3

Access	Session Manager or controlled SSH, no shared keys	EC2 Instance Connect, session logging to S3
Identity	Instance role with least privilege, no static keys on instances	Permission boundaries, IAM Access Analyzer reviews
Metadata	**IMDSv2 required**, hop limit 1 unless needed	Disable IMDS where unused, SCP enforcement
Storage	**Encrypted EBS** (encryption by default), snapshots/backups	Cross-Region copies, AWS Backup Vault Lock
Images	Hardened, patched, scanned AMIs, no secrets inside	EC2 Image Builder pipeline, AMI deprecation
Patching	A defined patch process (rebuild AMIs or Patch Manager)	Maintenance windows, canary rollouts
Availability	Multiple AZs behind a load balancer or Auto Scaling for production services	Cross-Region DR, capacity reservations
Monitoring	Status check alarms, CPU/memory/disk alarms (agent), logs shipped off-box	Detailed monitoring, Compute Optimizer, dashboards
Governance	Tags, IaC, CloudTrail, AWS Config rules	SCPs, Service Catalog
Threat detection	GuardDuty enabled	Inspector, Security Hub, runtime agents
Cost	Tags, budgets, orphaned volume cleanup	Savings Plans, Spot, Graviton

17.2 Least privilege and secrets

1. Separate roles per workload. Scope S3, KMS and Secrets Manager permissions to specific ARNs.
2. Get secrets at runtime from **Secrets Manager** or **Parameter Store**, never from AMIs, user data or Git.
3. Restrict `iam:PassRole` and `ec2:RunInstances` for human and CI roles. Use tags and conditions.
4. Use SCPs to require IMDSv2 (`ec2:MetadataHttpTokens`), block public AMI sharing and deny disabling EBS encryption defaults.

17.3 Network and access controls

1. App instances in private subnets behind an ALB. Databases in isolated subnets.
2. Admin by Session Manager. If a bastion is unavoidable, restrict by source IP, use MFA and log sessions.
3. Enable VPC flow logs and GuardDuty.
4. Review SGs regularly. AWS Config rules such as `restricted-ssh` and `ec2-instance-no-public-ip` help.

17.4 High availability and recovery

1. **Single instances are single points of failure.** For production use at least two instances in different AZs behind a load balancer, managed by an Auto Scaling group.
2. Store state outside the instance (RDS, DynamoDB, S3, EFS, ElastiCache).
3. Enable **auto recovery** for unavoidable singletons, with alarms and backups.
4. Take **EBS snapshots/AMIs** on a schedule (DLM or AWS Backup) and **test restores**.
5. For Regional failure: copy AMIs and snapshots cross-Region, keep IaC ready and rehearse the cutover.

17.5 Rollback and failure recovery

1. Change through **launch template versions**, with controlled rollout and rollback to the previous version.
2. Before risky changes on a stateful instance: snapshot or AMI.
3. Prefer **replace over repair**: if an instance is broken, rebuild from the AMI and restore data.

17.6 Monitoring, logging and alerting

1. CloudWatch agent for memory, disk, inodes and logs. Retention on log groups.
2. Alarms in section 14.5, routed to on-call.
3. CloudTrail for API actions, Config for drift, EventBridge for state changes and scheduled events.

17.7 Scalability and performance

1. Right-size from data. Use ELB request-based scaling for web tiers.
2. Choose storage by workload (gp3 default, io2 for demanding databases) and check instance EBS bandwidth.
3. Use placement groups only where latency demands it.
4. Keep AMIs lean and boot fast so scale-out is quick.

17.8 Cost optimisation

1. See section 5.3. Review monthly with Cost Explorer and tags.
2. Delete orphaned volumes, old snapshots and unused Elastic IPs.
3. Savings Plans for the steady baseline, Spot for fault-tolerant work, schedule dev environments off.

17.9 Backup and recovery

1. Snapshots with retention and cross-account copies for critical data.
2. Application-consistent backups for databases (native tools or quiesce).
3. Restore drills measure the real recovery time.

17.10 Compliance considerations

1. Encryption at rest and in transit, access logging, patch evidence, inventory (SSM Inventory, Config) and least privilege cover most frameworks (PCI DSS, HIPAA, SOC 2, ISO 27001).
2. Use Dedicated Hosts or Instances where licences or regulations require physical isolation.
3. Confirm Regions and services against the relevant AWS compliance documentation (AWS Artifact).

---

18. Comparison Tables

18.1 Stop vs reboot vs terminate vs hibernate

	Reboot	**Stop**	**Hibernate**	**Terminate**
Instance remains	Yes	Yes	Yes	**No**
Instance store data	Kept	**Lost**	Lost	Lost
EBS root and data	Kept	Kept	Kept	Root deleted by default
Public IPv4 (non-EIP)	Same	**New on start**	New on start	Gone
Compute billing	Yes	No	No	No
RAM state	Lost	Lost	**Saved**	Lost

18.2 EBS vs instance store vs EFS vs S3

	EBS	**Instance store**	**EFS**	**S3**
Type	Block	Block (local)	Network file system (NFS)	Object
Scope	One AZ	One host	Regional, many instances	Regional
Persistence	Persistent	Temporary	Persistent	Persistent
Shared across instances	Mostly no (Multi-Attach limited)	No	**Yes**	Yes (via API)
Use	OS, databases	Cache, scratch	Shared files	Objects, backups, static content

18.3 Security group vs NACL

(See section 9.1.)

18.4 Public IP vs Elastic IP vs private IP

	Private IP	**Public IPv4 (auto-assigned)**	**Elastic IP**
Reachable from internet	No	If routes and rules allow	If routes and rules allow
Survives stop/start	**Yes**	**No**	**Yes**
Movable between instances	No (ENI can move)	No	**Yes**
Cost	Free	Charged (verify)	Charged (verify)

18.5 Access methods

	SSH key pair	EC2 Instance Connect	Session Manager
Inbound port needed	22	22	**None**
Long-lived credential	Private key	No (60-second key)	No (IAM)
Audit	Weak (server logs)	CloudTrail	CloudTrail + session logs
Works for private instances	Via bastion or VPN	Via endpoint	**Yes** (with endpoints or NAT)

18.6 Purchasing options

	On-Demand	Savings Plans / RIs	Spot	Dedicated
Commitment	None	1 or 3 years	None	Varies
Discount	None	Significant	Largest	None or varies
Interruptible	No	No	**Yes (2-minute notice)**	No
Use	Variable load	Steady baseline	Fault-tolerant	Licences, compliance

18.7 EC2 vs containers vs Lambda

	EC2	ECS/EKS on EC2	Fargate	Lambda
You manage	OS, patching, scaling	Nodes and containers	Containers only	Code only
Best for	Full control, special software	Container platforms	Containers without servers	Short event-driven code
Cold start	Minutes	Pod/node scale	Seconds to a minute	Milliseconds to seconds

18.8 Common interview traps

1. **"A public IP makes it reachable."** Not without IGW routes and SG/NACL rules.
2. **"Stop and start keeps my public IP."** Only an Elastic IP is retained.
3. **"An IAM role is like an SSH key."** No: the role is for AWS API access, the key is for OS login.
4. **"Snapshots are consistent backups of my database."** Crash-consistent only unless coordinated.
5. **"I can shrink an EBS volume."** No.
6. **"EBS volumes can attach across AZs."** No. Snapshot and restore into the other AZ.
7. **"Instance store survives stop."** No.
8. **"A security group can deny traffic."** No. Only NACLs have deny rules.
9. **"Termination protection stops Auto Scaling from terminating."** No. Use scale-in protection.
10. **"Memory and disk metrics come with EC2 by default."** No. Install the CloudWatch agent.
11. **"User data runs on every boot."** Only first boot by default.
12. **"Encrypting an existing volume in place is possible."** Snapshot, copy with encryption, create a new volume.

---

19. Enterprise Reference Architecture

19.1 Production web tier on EC2

```
Route 53 -> CloudFront + WAF (optional) -> ALB (public subnets, 3 AZs, TLS via ACM)
                                          |
                                Target group (health check /healthz)
                                          |
          +---------------- Auto Scaling group (private subnets, 3 AZs) -------------------+
          | Launch template: golden AMI, IMDSv2 required, encrypted gp3, instance profile  |
          | Access: Session Manager only. SG: app port from ALB SG only                    |
          | Agents: SSM, CloudWatch agent, Inspector. Patching by AMI rebuild + refresh    |
          +--------------------------------------------------------------------------------+
               |                     |                          |
        RDS Multi-AZ           ElastiCache                S3 (assets, artifacts)
   Outbound: NAT per AZ + VPC endpoints (S3, SSM, ECR, Secrets Manager, CloudWatch Logs)
   Observability: CloudWatch alarms/logs, flow logs, CloudTrail, Config, GuardDuty
   Backup: DLM/AWS Backup for EBS, AMI copies cross-Region
```

19.2 Image and release flow

```
Git (Packer/Image Builder template + app) -> CI builds AMI -> scan + tests
   -> publish AMI ID (SSM parameter) -> Terraform updates launch template version
   -> instance refresh with health checks and auto rollback -> monitor -> deprecate old AMIs
```

19.3 Where EC2 appears in other designs

1. **EKS/ECS worker nodes:** managed node groups are Auto Scaling groups of EC2 instances.
2. **Bastion replacement:** SSM Session Manager with VPC endpoints.
3. **CI runners:** Spot instances in an ASG that scale on queue depth.
4. **Lift-and-shift:** servers migrated to EC2 with AWS Application Migration Service.

19.4 Design decisions to explain in an interview

1. Why private subnets with ALB in front, and how admins get access without SSH.
2. How you patch (golden AMI and rolling replace vs in-place), and how you roll back.
3. How you protect role credentials (IMDSv2, least privilege, hop limit).
4. How data is protected (EBS encryption, snapshots, state outside the instance).
5. How you handle failure at instance, AZ and Region levels.
6. How you control cost (right-sizing, Savings Plans, Spot, cleanup).

---

20. Interview Questions and Answers

> Answer pattern: **definition → how it works → trade-off or example → follow-up**. Follow-up questions a senior interviewer may ask are shown as **Follow-up**.

20.1 Basic concepts

Q1. What is Amazon EC2 and why do we use it?

1. EC2 provides virtual servers (instances) in AWS. You choose OS (AMI), CPU and memory (instance type), storage and networking.
2. Benefits: elasticity, scalability, many instance shapes, automation (CLI, SDK, Terraform), multi-AZ availability and security controls.
3. AWS manages hardware and the hypervisor. **You manage the guest OS, patching, software and access.**
4. Example: host a Java app, with a role for S3 access and only the load balancer allowed to reach port 8080.
5. **Follow-up:** "When not EC2?" Short event-driven code (Lambda), containers without server management (Fargate), managed databases (RDS).

Q2. What are the steps and components involved in launching an EC2 instance?

1. Choose an **AMI**, an **instance type**, **network** (VPC, subnet, public IP), **security group**, **IAM role**, **storage** (EBS), **user data** and access method (key pair or SSM).
2. The instance goes `pending` then `running` in one subnet and AZ.
3. Components: AMI, instance type, EBS, key pair, security group, VPC and subnet, IAM role and instance profile, user data, Elastic IP, ENI, launch template, Auto Scaling group.
4. In production, launch from a **launch template** in an **Auto Scaling group**, not by hand.

Q3. What is an AMI, and why use a custom AMI?

1. A template with the root volume (snapshot), launch permissions and block device mapping, used to start instances. It is **Regional**.
2. Types: AWS-provided, Marketplace, community (untrusted until reviewed), custom.
3. Custom AMIs give consistency, faster boot, hardening and agents pre-installed.
4. No secrets in AMIs. Build them in a pipeline with scans, patch by rebuilding.
5. **Follow-up:** "Deregistering an AMI deleted my snapshots?" No. Delete the snapshots separately, or they keep costing money.

Q4. What are the EC2 instance families and how do you choose a type?

1. General purpose (T, M), compute optimised (C), memory optimised (R, X), storage optimised (I, D), accelerated (P, G) and others.
2. Choose by workload (CPU, memory, I/O, network), measured utilisation, network and EBS bandwidth limits, architecture (x86 or Arm), cost.
3. Right-size with CloudWatch and Compute Optimizer. Prefer newer generations and test Graviton.
4. Naming example: `m7i.large` = family m, generation 7, Intel, size large.
5. **Follow-up:** "Why not just pick the biggest?" Cost, and it hides inefficiency. Bigger instances also do not fix a single-threaded bottleneck.

Q5. What is the difference between stop, reboot, terminate and hibernate?

1. **Reboot:** OS restart, same host, same IPs, instance store kept.
2. **Stop:** powers off an EBS-backed instance, no compute charge, EBS still billed, instance store lost, non-Elastic public IP lost.
3. **Terminate:** deletes the instance. Root volume is deleted if `DeleteOnTermination` is true (default for root).
4. **Hibernate:** saves RAM to the encrypted root volume and stops, resumes with state.
5. Auto Scaling replaces stopped or terminated members. Use Standby instead of stopping.
6. **Follow-up:** "Does termination protection stop ASG scale-in?" No. Use scale-in protection.

Q6. EBS vs instance store?

1. **EBS:** network block storage in one AZ, persistent, snapshots, resizable, encryption with KMS. Use for OS and data.
2. **Instance store:** local to the host, very fast, **temporary** (lost on stop, hibernate, terminate, host failure; kept on reboot). Use for cache and scratch.
3. Do not keep irreplaceable data on instance store.
4. **Follow-up:** "How do you move an EBS volume to another AZ?" Snapshot it and create a new volume from the snapshot in the other AZ.

Q7. Explain private, public and Elastic IP addresses.

1. **Private IPv4:** from the subnet, constant for the instance's life.
2. **Public IPv4:** auto-assigned, **changes on stop/start**.
3. **Elastic IP:** static public IPv4 you own and can re-associate. Public IPv4 addresses are charged (verify).
4. A public IP alone does not make the instance reachable. Routes and SG/NACL rules must allow it.
5. Prefer a load balancer and DNS names over fixed instance IPs.

Q8. Public vs private subnet, and IGW vs NAT gateway?

1. A subnet is **public** when its route table sends `0.0.0.0/0` to an internet gateway.
2. **IGW:** two-way internet access for resources with public IPs. **NAT gateway:** outbound-only access for private instances. It sits in a public subnet and costs per hour and per GB.
3. Typical layout: ALB in public subnets, app instances in private subnets, databases in isolated subnets.
4. Use VPC endpoints to reach AWS services without NAT, and one NAT per AZ for resilience.

Q9. Security groups vs network ACLs?

1. SG: instance-level, **stateful**, allow rules only. NACL: subnet-level, **stateless**, allow and deny, numbered rules.
2. Reference SGs between tiers instead of IP ranges.
3. NACLs need ephemeral ports (1024–65535) for return traffic.
4. Neither is a substitute for authentication. Do not expose SSH or databases to the internet.
5. **Follow-up:** "How would you block one abusive IP?" NACL deny rule (or WAF). SGs cannot deny.

Q10. IAM role vs key pair?

1. The **IAM role** (through an instance profile) gives the software on the instance temporary **AWS API permissions**.
2. The **key pair** authenticates a **person** to the OS over SSH.
3. They solve different problems. Use roles instead of access keys on instances. Prefer SSM over SSH keys.
4. Verify the identity with `aws sts get-caller-identity` (expect `assumed-role/<role>/<instance-id>`).

20.2 Intermediate concepts

Q11. Explain EBS volume types and snapshots.

1. **gp3** (default SSD), gp2 (legacy), **io2** (provisioned IOPS), st1 and sc1 (HDD, not bootable).
2. IOPS and throughput are also limited by the instance's EBS bandwidth.
3. **Snapshots** are incremental, Regional, point-in-time backups. Restore creates a new volume (any AZ in the Region). Lazy loading makes first reads slow, so use Fast Snapshot Restore if needed.
4. Automate with DLM or AWS Backup. Snapshots are crash-consistent. Use native backup or quiesce for databases.
5. Enable **encryption by default** per Region.

Q12. How do you increase an EBS volume without downtime?

1. `modify-volume --size <new>` (Elastic Volumes) works while attached.
2. Then grow inside the OS: `growpart` for the partition, `xfs_growfs` (XFS) or `resize2fs` (ext4) for the file system. Verify with `df -hT`.
3. There is a cooldown (about 6 hours) before another modification, and **volumes cannot shrink**.
4. Snapshot first as a safety net.

Q13. How do key pairs and SSH work, and how do you troubleshoot SSH failures?

1. The public key is placed in `authorized_keys` at launch. You hold the private key (`chmod 400`).
2. Username depends on the AMI (`ec2-user`, `ubuntu`).
3. Timeout: SG, NACL, route, public IP, instance state. Refused: `sshd` or host firewall. Permission denied: user or key.
4. Use `ssh -vvv`, status checks and console output. A full disk can also break SSH.
5. A lost key cannot be recovered. Use SSM, Instance Connect, or a rescue instance to add a new key.

Q14. What is user data and how do you troubleshoot it?

1. A script or cloud-config run by cloud-init, **once at first boot, as root** by default. Size limit is about 16 KB.
2. Needs `#!`. Output is in `/var/log/cloud-init-output.log`. Use `cloud-init status --long`.
3. Not secret: never put passwords or keys in it. Fetch secrets using the role.
4. Failures are usually no outbound internet, package manager mismatch or script errors. Bake heavy installs into the AMI.

Q15. What is IMDSv2 and why enforce it?

1. The instance metadata service at `169.254.169.254` returns instance data and role credentials.
2. IMDSv1 answers plain GETs and is exploitable through SSRF. **IMDSv2** requires a session token obtained with a `PUT`, which blocks most SSRF.
3. Enforce with `HttpTokens=required` in launch templates, an account default and an SCP condition.
4. Hop limit 1 prevents containers from reaching the node role. Use task or pod roles for containers.
5. **Follow-up:** "Does IMDSv2 make SSRF impossible?" No. It raises the bar. Also fix the vulnerable app and limit role permissions.

Q16. What are launch templates and why use them?

1. Versioned launch settings: AMI, type, SGs, profile, EBS, user data, metadata options, Spot options and tags.
2. Roll out changes by creating a new version and an instance refresh. Roll back by selecting the older version.

3. They replace launch configurations (deprecated).
4. Put secure defaults (IMDSv2, encrypted EBS) in the template.
5. See `ec2-auto-scaling.md` for rollout details.

Q17. Explain EC2 purchasing options.

1. **On-Demand:** no commitment. **Savings Plans / RIs:** 1 or 3 year commitment for discounts. **Spot:** spare capacity at a deep discount, interruptible with 2 minutes' notice. **Dedicated Hosts/Instances:** physical isolation for licences or compliance. **Capacity Reservations:** guaranteed capacity in an AZ.
2. Use Savings Plans for the steady baseline, On-Demand for variable load, Spot for fault-tolerant work.
3. Review with Cost Explorer and Compute Optimizer.
4. **Follow-up:** "Compute vs EC2 Instance Savings Plan?" Compute is flexible across families and Regions. EC2 Instance gives a larger discount but ties to a family and Region.

Q18. What are EC2 status checks and how does recovery work?

1. **System check:** AWS host and network. **Instance check:** guest OS and network configuration.
2. System failure: stop and start (new host) or auto recovery. Instance failure: reboot, console output, or repair with a rescue instance.
3. Auto recovery keeps the instance ID, private IP, EIP and volumes. Memory is lost.
4. Alarm on `StatusCheckFailed_System` with the recover action. In an ASG the group replaces unhealthy instances.

Q19. How do burstable (T) instances work?

1. They have a baseline CPU and earn credits when idle, spending them to burst.
2. When credits are out, **standard** mode throttles to baseline. **Unlimited** mode bursts and charges for surplus credits.
3. Watch `CPUCreditBalance`.
4. Good for spiky low-average loads. Use M or C types for steady load.

Q20. What is an ENI, and what do you use multiple ENIs for?

1. A virtual network card with private IPs, optional public/Elastic IP, MAC and security groups.
2. Multiple ENIs allow separate management and data networks, dual-homed appliances and **IP failover** (move the ENI to a standby instance).
3. ENIs are AZ-bound. The number allowed depends on instance type.
4. Source/destination check should be disabled only for NAT or firewall appliances.

20.3 Advanced technical questions

Q21. What are the Nitro System, ENA and EBS-optimised instances, and why do they matter?

1. **Nitro** offloads networking, storage and security to dedicated hardware, giving near bare-metal performance and strong isolation.
2. **ENA** is the high-performance network interface. **EBS-optimised** gives dedicated bandwidth to EBS (on by default on current types).
3. Every instance type has **EBS bandwidth and network limits**, so a fast volume on a small instance can still be capped.
4. Old AMIs may lack ENA or NVMe drivers and fail on Nitro types.

Q22. How does hibernation work and what are the limits?

1. RAM contents are written to the **encrypted root EBS volume**, then the instance stops. On start, RAM is restored and processes continue.
2. Must be enabled at launch, with a supported type and AMI, an encrypted root volume large enough for RAM, and RAM within limits. Verify the current limits and maximum duration.
3. Use for long-initialisation applications. Not for instance-store workloads (data is lost).

Q23. Explain placement groups.

1. **Cluster:** low latency, high throughput, one AZ (higher correlated-failure risk). **Spread:** each instance on distinct hardware, limited per AZ. **Partition:** groups on separate racks for large distributed systems.
2. Launch related instances together and use a single instance type to reduce capacity errors in cluster groups.
3. Use only when needed. They can cause `InsufficientInstanceCapacity`.

Q24. How do you build and maintain AMIs, and how do you patch a fleet?

1. Pipeline: base AMI -> Image Builder or Packer -> tests and scans -> tag and version -> share/copy -> new launch template version -> instance refresh -> deprecate old AMIs.
2. **Immutable approach:** replace instances with new AMIs instead of patching in place. Roll back by selecting the previous version.
3. For long-lived servers: SSM Patch Manager with maintenance windows and baselines.
4. Never bake secrets. Track AMI IDs via SSM parameters.

Q25. How do you run workloads on Spot safely?

1. Use stateless or checkpointed work, with retries and graceful shutdown.
2. Handle the **2-minute interruption notice** (metadata or EventBridge) and **rebalance recommendations**.
3. Diversify across many types and AZs, use `price-capacity-optimized`, run via Auto Scaling or EC2 Fleet with an On-Demand base.
4. Do not run unrecoverable singletons on Spot.

Q26. How do you move or recover an instance across AZs or Regions?

1. **Across AZs:** create an AMI or snapshot, launch or create volumes in the other AZ. Reassign Elastic IP or DNS.
2. **Across Regions:** copy the AMI (and snapshots) to the target Region (with a KMS key there), launch with IaC, update DNS.
3. Keep state outside instances where possible. For DR, pre-copy AMIs and snapshots and rehearse the cutover.
4. Instance IDs and private IPs change in the new location.

Q27. What do you do about `InsufficientInstanceCapacity`?

1. Retry in another AZ or with another instance type or size (mixed instances policy).
2. Avoid placement groups that need co-location if not necessary.
3. For critical needs use **On-Demand Capacity Reservations** (and Savings Plans for the cost).
4. Check quotas separately (`VcpuLimitExceeded` is a different problem).

20.4 Scenario-based and production troubleshooting questions

Q28. You cannot SSH to an instance. How do you troubleshoot?

1. Is the instance running and passing status checks?
2. Timeout or refused or permission denied? That tells you network vs service vs key.
3. Network: public IP, IGW route, SG inbound 22 from your current IP, NACL both ways.
4. Auth: username, key and permissions.
5. If Session Manager works, check `sshd`, disk, and `/var/log/secure`.
6. Fix and consider moving to Session Manager with no inbound 22.

Q29. A production instance is at 100% CPU. What do you do?

1. Check what changed (release, traffic) and CloudWatch trends.
2. On T types, check `CPUCreditBalance`.
3. `top` / `vmstat` : user, system or iowait? Which process? Unknown processes need a security check.
4. Short-term: restart or throttle the process, scale out. Fix: code or query, right-size, switch family.
5. Add alarms and load tests.

Q30. The disk is full. How do you handle it?

1. `df -hT` and `df -ih` (space and inodes).
2. `du` for the largest directories, `lsof +L1` for deleted-but-open files.
3. Free space safely (rotate logs, prune), restart the process holding deleted files.
4. Grow the EBS volume and file system only if data is legitimate.
5. Add `logrotate`, agent alarms and separate data volumes.

Q31. An instance shows a failed status check. What now?

1. System or instance check?
2. System: stop and start, or auto recovery. Instance: console output, reboot, then a rescue instance if the root volume or

`fstab` is broken.
3. Snapshot before repairing.
4. Prevent with `nofail` and UUIDs in `fstab`, disk and memory alarms and immutable replacement.

Q32. A private-subnet instance cannot reach the internet or SSM. Why?

1. Check the route table for `0.0.0.0/0` to a NAT gateway (in a public subnet with an IGW route) or the presence of VPC endpoints.
2. SG outbound 443, NACL both directions, DNS support.
3. For SSM: agent, instance profile with `AmazonSSMManagedInstanceCore`, endpoints for `ssm`, `ssmmessages`, `ec2messages` with private DNS and an SG allowing 443.
4. Use Reachability Analyzer and flow logs.

Q33. The app on EC2 gets AccessDenied from AWS. How do you debug?

1. Run `aws sts get-caller-identity` as the app. Expect the instance role. Static keys in env vars may override it.
2. Check the instance profile is attached and the container can reach IMDS (hop limit) or uses a task or pod role.
3. Read the CloudTrail denial. Check identity policy, boundary, SCP, resource policy and KMS key policy.
4. Add the narrowest permission, not `*`.

20.5 Security and best-practice questions

Q34. How do you harden an EC2 instance and fleet?

1. Private subnets, SG from the load balancer only, no public admin ports.
2. Session Manager for access, no shared keys.
3. IMDSv2 required, least-privilege role, no static keys.
4. Encrypted EBS by default, hardened and scanned AMIs, patching process.
5. GuardDuty, Inspector, CloudTrail, Config, flow logs.
6. IaC, tags, SCP guardrails.

Q35. Why prefer Session Manager over SSH?

1. No inbound ports, no bastion, no long-lived keys.
2. IAM-controlled, MFA-capable, CloudTrail audit and session logs to S3/CloudWatch.
3. Works for private instances, supports port forwarding and Run Command.
4. Needs the SSM agent, a role and a network path (NAT or endpoints).
5. SSH is still needed by some tools. Restrict and log it if used.

Q36. How can an attacker steal EC2 role credentials, and how do you prevent it?

1. An app with SSRF is tricked into fetching `169.254.169.254` (IMDSv1) and returns the role credentials.
2. Prevent: **IMDSv2 required**, hop limit 1, least-privilege role, fix the app, WAF, network egress limits.
3. Detect: GuardDuty findings for credentials used from outside AWS or unusual API calls.
4. Respond: quarantine the instance, **revoke the role's active sessions**, investigate with CloudTrail and rotate secrets.

20.6 Follow-up questions a senior interviewer may ask

1. "How would you migrate 50 servers into EC2 with minimal downtime?" Discover and group them, use AWS Application Migration Service for continuous replication, test cutover in a separate subnet, cut over in a window, keep rollback.
2. "How do you stop engineers from launching public instances with open SGs?" SCPs, AWS Config rules with auto-remediation, Service Catalog, IaC reviews and guardrail alarms.
3. "Your costs doubled this month. How do you investigate?" Cost Explorer by tag and service, look for new instances, NAT data transfer, orphaned EBS and snapshots, public IPv4 charges, Spot to On-Demand fallbacks and unusual activity (GuardDuty).
4. "How do you give a third party temporary access to one instance?" Time-bound IAM role with Session Manager restrictions by tag, session logging, no keys shared, automatic expiry.
5. "How would you design for an AZ failure?" Instances in at least two AZs behind a load balancer with an ASG, state in multi-AZ services, enough N-1 capacity, tested failover.
6. "How do you handle OS patching with zero downtime?" Rebuild AMIs and rolling instance refresh behind a load balancer with health checks, or Patch Manager in batches with deregistration delay.
7. "When would you choose EC2 over ECS or EKS?" Special OS or kernel needs, licence constraints, a small number of long-

running stateful servers, GPUs with custom stacks, or when containerisation is not worth the effort.

---

21. Interview Answer Framework

Use this for any scenario question. Say the steps aloud.

21.1 The eight-step framework

Step	What you say	EC2-specific examples
1. Understand problem and impact	What is failing, who is affected, since when, what changed	One instance or the fleet? Release, patch, config change, AWS event?
2. Check monitoring, logs, metrics, events	Evidence first	Status checks, CloudWatch (CPU, credits, memory, disk), console output, CloudTrail, scheduled events
3. Verify architecture and configuration	Draw the chain	Client -> DNS -> ALB -> SG -> instance -> role -> dependencies (RDS, S3). Subnet, route, NACL
4. Narrow down root causes	Rank hypotheses	Network, auth, resource exhaustion, bad deployment, AWS host issue
5. Run safe diagnostics	Read-only first	`describe-instance-status`, `top`, `df`, `journalctl`, `cloud-init status`, flow logs
6. Apply the safest fix and validate	Smallest change, with rollback	Fix SG, free disk, restart service, replace instance from AMI, snapshot first
7. Communicate and document	Status, timeline, root cause	Incident updates, post-incident review, action items
8. Prevent recurrence	Alerts and automation	Agent alarms, IaC, immutable AMIs, SSM, Config rules, runbooks

21.2 EC2 adaptations

1. **Unreachable:** instance state and checks -> network path (SG, NACL, route, IP) -> OS (sshd, disk, memory) -> auth.
2. **Slow:** CPU, memory, disk, network metrics -> credits (T types) -> top processes -> application logs -> dependencies.
3. **Boot or launch failure:** console output -> cloud-init logs -> network for installs -> quotas and capacity -> IAM/KMS.
4. **Security incident:** quarantine SG -> revoke role sessions -> preserve evidence -> CloudTrail -> rebuild clean.
5. **Cost spike:** Cost Explorer by tag -> idle and orphaned resources -> NAT and data transfer -> purchasing mix.

21.3 Worked example (about 90 seconds)

> **Question:** "You cannot SSH to a production EC2 instance. What do you do?"
>
> "First I'd confirm the impact: is the application still serving traffic, and did anything change, like a security group edit, a new key or a deployment? Then I'd check evidence: the instance state, status checks and the console output.
>
> I'd separate network from authentication. A timeout points to the network: security group inbound 22 from my current IP, the NACL in both directions, a public IP and an internet gateway route, or the instance being down. 'Connection refused' points to `sshd` or a host firewall. 'Permission denied' means the network works and the username or key is wrong. I'd run read-only checks: `describe-instance-status`, `describe-security-groups`, the route table, and `ssh -vvv`.
>
> If Session Manager works, I'd use it to check `sshd`, disk space and `/var/log/secure`. Say the security group only allowed the office IP; the safest fix is a narrow, time-boxed rule for my IP. Longer term I'd move to Session Manager with no inbound SSH, manage the rules in Terraform and alert on status checks."

21.4 Delivery tips

1. State assumptions ("assuming Amazon Linux 2023 in a private subnet behind an ALB").
2. Walk the chain from client to instance to dependencies.
3. Prefer read-only diagnostics first and say so.
4. Mention trade-offs (cost vs availability, repair vs replace, SSH vs SSM).
5. Do not claim you built something you did not. Use "in a design like this I would…" for hypotheticals, and use your real project experience only where it is genuinely yours.

22. Quick Revision

22.1 Key points to remember

1. EC2 = virtual servers. AWS owns hardware and hypervisor. **You own the OS, patching, software and access.**
2. Instance = AMI + type + subnet/AZ + SG + role + storage + user data. **Subnet = one AZ.**
3. **Stop** loses instance store and non-Elastic public IP. **Reboot** keeps both. **Terminate** is permanent.
4. **EBS** is persistent, per AZ, snapshot-able. Snapshots are Regional. **Volumes grow, never shrink.**
5. **Security groups** are stateful allow-only. **NACLs** are stateless with deny.
6. **IAM role** = AWS API permissions. **Key pair** = OS login. Prefer **Session Manager**.
7. **IMDSv2 required.** Hop limit 1. Credentials come from IMDS.
8. User data runs **once** at first boot and is **not secret**.
9. Memory and disk metrics need the **CloudWatch agent**.
10. System status check = AWS side (stop/start, auto recover). Instance status check = OS side (console output, rescue).
11. T instances throttle when credits end. Use M or C for steady load.
12. Production = ASG + ALB across AZs, golden AMIs, private subnets, state outside the instance.

22.2 Important commands

```
aws ec2 describe-instances --instance-ids <id>
aws ec2 describe-instance-status --instance-ids <id> --include-all-instances
aws ec2 get-console-output --instance-id <id> --latest --output text
aws ec2 modify-instance-metadata-options --instance-id <id> --http-tokens required
aws ec2 create-snapshot --volume-id <vol-id>
aws ec2 modify-volume --volume-id <vol-id> --size <new-size>
aws ssm start-session --target <instance-id>
aws sts get-caller-identity
```

```
lsblk; df -hT; df -ih; findmnt
sudo growpart /dev/nvme0n1 1 && sudo xfs_growfs -d /
sudo cloud-init status --long; sudo less /var/log/cloud-init-output.log
top; free -h; vmstat 1 5; iostat -xz 1 5; ss -tulpn; journalctl -p err -b
```

22.3 Commonly confused concepts

Pair	Difference
AMI vs instance	Image vs running server
EBS vs instance store	Persistent vs temporary
Public IP vs Elastic IP	Changes on stop vs static
SG vs NACL	Instance, stateful, allow vs subnet, stateless, allow/deny
IGW vs NAT gateway	Two-way for public IPs vs outbound-only for private
IAM role vs key pair	AWS API access vs OS login
System vs instance status check	AWS host vs guest OS
Stop vs terminate	Keeps instance and EBS vs deletes instance
Snapshot vs AMI	Volume backup vs launchable image (built from snapshots)
IMDSv1 vs IMDSv2	Plain GET vs token-based session
User data vs AMI	Runtime bootstrap vs pre-baked
Savings Plan vs Spot	Commitment discount vs interruptible discount

22.4 Five-minute revision notes

1. **Launch right:** hardened AMI, private subnet, SG from ALB only, least-privilege role, IMDSv2, encrypted gp3, no key pair, Session Manager.

2. **Operate:** CloudWatch agent (memory, disk), status check alarms, logs shipped, snapshots, patch by rebuilding AMIs.
3. **Troubleshoot:** state and status checks -> network (SG, NACL, route) -> OS (disk, memory, CPU, services) -> identity and dependencies.
4. **Recover:** auto recovery for hardware faults, rescue instance for OS faults, replace from AMI for everything else.
5. **Save money:** right-size, Savings Plans, Spot, gp3, Graviton, delete orphans, schedule non-prod.

22.5 Quick-reference cheat sheet

Need	Use
Shell without opening ports	Session Manager
AWS permissions for the app	Instance role (least privilege)
Same config on many servers	Launch template + ASG + golden AMI
Survive AZ failure	Multiple AZs behind a load balancer
Grow a disk	`modify-volume` then `growpart` and `xfs_growfs`
Move a volume to another AZ	Snapshot, create volume in the other AZ
Protect credentials from SSRF	IMDSv2 required, hop limit 1
Private instance needs internet	NAT gateway or VPC endpoints
Cheap fault-tolerant compute	Spot with diversification
Guarantee capacity	On-Demand Capacity Reservation

22.6 Ten rapid-fire questions (with short answers)

1. **Does a public IP alone make an instance reachable?** No. Routes and SG/NACL must allow it.
2. **What survives a stop?** EBS data and private IP. Not instance store or a non-Elastic public IP.
3. **Can an SG deny traffic?** No. Only NACLs deny.
4. **Can you shrink an EBS volume?** No.
5. **Which status check does stop/start fix?** The system status check.
6. **Where do role credentials come from?** The instance metadata service.
7. **Does user data run on every boot?** No, only the first boot by default.
8. **What blocks SSRF credential theft?** IMDSv2 (plus least privilege).
9. **Which metrics need the CloudWatch agent?** Memory and disk space.
10. **First command when you cannot SSH?** `aws ec2 describe-instance-status`, then check the SG and route.

22.7 Final interview readiness checklist

Tick each item only if you can explain it **without notes**.

- [ ] I can draw the EC2 launch flow and the ALB + private subnet + RDS architecture.
- [ ] I can explain AMIs, golden AMI pipelines and why not to bake secrets.
- [ ] I can choose instance families and explain burstable credits, Graviton and Nitro.
- [ ] I can compare On-Demand, Savings Plans, Spot, Dedicated and Capacity Reservations.
- [ ] I can compare stop, reboot, terminate and hibernate, and what each keeps.
- [ ] I can explain EBS types, snapshots, resizing and instance store.
- [ ] I can explain public, private and Elastic IPs, IGW vs NAT, ENIs and placement groups.
- [ ] I can compare security groups and NACLs.
- [ ] I can explain key pairs, Session Manager and Instance Connect.
- [ ] I can explain user data, cloud-init, IMDSv2 and instance roles.
- [ ] I can explain status checks, auto recovery and the CloudWatch agent.
- [ ] I can read and write Terraform for a hardened instance and a launch template.
- [ ] I can troubleshoot "cannot SSH", "disk full", "no internet in private subnet", "status check failed", "AccessDenied", "user data failed", and a compromised instance using the eight-step framework.
- [ ] I have rehearsed at least three scenarios aloud.

22.8 Official documentation (verify current details here)

- What is Amazon EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html
- Instance types: https://docs.aws.amazon.com/ec2/latest/instancetypes/instance-types.html
- Amazon EBS: https://docs.aws.amazon.com/ebs/latest/userguide/what-is-ebs.html

- Instance metadata (IMDSv2): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html
- IAM roles for EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-roles-for-amazon-ec2.html
- Security groups: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
- Session Manager: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html
- Status checks: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/monitoring-system-instance-status-check.html
- Purchasing options: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-purchasing-options.html
- Troubleshooting EC2: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-troubleshoot.html

> Some of these URLs may be reorganised by AWS over time. If a link moves, search the title on the AWS documentation site.